

2. ATALA

Datu Zientziaren Stack-a

EGILEA

Aitor Aretxaga Agirre

ZENTROA

Uni Eibar-Ermua

EDIZIOA

v1.1 (Laneki — FP Euskadi materialean oinarritua)

IKASTAROA

2026 — 2027

Datu Zientziaren Stack-a

Sarrera

Imajina ezazu zure bigarren astea enpresan. Lehen astean ingurunea prestatu duzu (Python, ingurune birtuala, Git), eta orain benetako lana iristen da. Proiektu-zuzendariak Excel fitxategi bat bidaltzen dizu posta elektronikoz: "Hemen daukazu azken hiru hilabeteetako salmenten erregistroa, 85.000 errenkada. Marketin-taldeak jakin nahi du zein hiritan saltzen dugun gehien, zein produktu-kategoria hazten ari den, eta ea ikus daitekeen joeraren bat grafiko batean. Bihar goizerako, ahal bada".

85.000 errenkada. Excel-ek bere onenak ematen ditu, baina kalkuluak motel doaz, formula konplexuek fitxategia blokeatzen dute, eta taula dinamikoak eskuz egitea ordu asko da. Gainera, datuetan akatsak daude: hiri batzuk minuskulaz idatzita ("bilbo"), beste batzuk maiuskulaz ("BILBO"), prezio batzuk hutsik, eta errenkada bikoiztuak. Zer egin?

Hau da datu-zientziaren stack-a sartzen den unea. Pythonen lerro gutxi batzuekin, 85.000 errenkada segundo gutxitan kargatzen dira (**Pandas**), hiri-izenak normalizatzen dira (**datu-garbigeta**), hiriko eta kategoriako batezbestekoak kalkulatzeko dira (**groupby**), eta joera bat grafiko profesional batean erakusten da (**Matplotlib/Seaborn**). Eskuz ordu asko hartuko zukeen lana 20 lerrotan eta segundo batzuetan egiten da, gainera errepika daitekeen modu batean: hileroko datu berriak iristen direnean, script bera exekutatu eta kito.

Gai honek datu-zientzialari batek egunero erabiltzen duen tresna-multzoa irakatsiko dizu: **NumPy** (zenbakizko kalkulu azkarra), **Pandas** (taulen kudeaketa), **Matplotlib eta Seaborn** (bistaratzeko), **DVC** (datuen bertsio-kontrola) eta merkatuko **plataforma komertzialak** (Azure ML, IBM Watson, Knime, SPSS). Hauek dira AAren eta Machine Learningaren oinarrian dauden tresnak: eredu bat entrenatu aurretik, datuak ulertu, garbitu eta prestatu behar dira, eta hori da, hain zuzen ere, datu-zientzialariaren denboraren %60-80 hartzen duen lana.

! Ideia nagusia Datu-zientziaren stacka ez da matematika hutsa: enpresaren datu nahasiak (Excel, CSV, datu-baseak) erabaki-iturri bihurtzeko tresna-multzoa da.

Datuak → Informazioa → Erabakia!

Eredu bat entrenatzea datua-informazio bihurtzeko azken urratsa da; aurretik datuak ulertu, garbitu eta prestatu behar dira, eta hori egiten du lana stack honek.

1. NumPy: Array-ak eta Eragiketa Bektorialak

Atal honetan **NumPy** liburutegiaren oinarriak landuko ditugu, datu-zientziaren ekosistema osoaren zutabe nagusia baita. NumPy gabe, Pandas-ek, scikit-learn-ek, TensorFlow-k edo PyTorch-ek ezingo lukete eraginkortasunez funtzionatu: denek NumPy array-ak erabiltzen dituzte barne-egitura gisa. Ikasiko duzu zergatik den hain garrantzitsua, nola sortu eta manipulatu array-ak, nola aplikatu eragiketa bektorialak `for` begiztak saihesteko, eta zer aukera eskaintzen dituen estatistikan eta algebra linealean. BOEko Errege Dekretuaren (RD 279/2021) IE1 ikaskuntza-emaizaren oinarria da hau: datu-zientziaren ekosistema teknologikoa menderatzea.

1.1 NumPy zer da eta zergatik erabili

NumPy (Numerical Python) Pythonen zenbakizko kalkulurako liburutegi nagusia da. Datu Zientziaren, Machine Learningaren eta Adimen Artifizialaren oinarri-oinarria da. Ia liburutegi guztiek NumPy arrayak erabiltzen dituzte barne-egitura gisa. Esan dezakegu NumPy datu-zientziaren *porlana* dela: ez duzu beti zuzenean ikusten, baina haren gainean dago dena eraikita.

Pythonen zerrenda arruntek malgutasun handia eskaintzen dute, baina zenbakizko kalkulurako ez dira eraginkorrak. Zergatik? Zerrenda batean elementu bakoitza Python-objektu independente bat da, memorian sakabanatuta dagoena; NumPy array batean, aldiz, elementu guztiak mota berekoak dira (adibidez, `float64` denak) eta memorian elkarren ondoan kokatzen dira, bloke trinko batean. Bloke trinko horrek bi abantaila ekartzen ditu: memoria gutxiago kontsumitzen da eta prozesatzailea askoz azkarrago ibiltzen da datuetan zehar.

```
import numpy as np
import time

# Denbora-konparaketa: 10 milioi elementuko zerrenda
N = 10,000,000

# Python zerrenda
lista = list(range(N))
hasiera = time.time()
emaitza = [x * 2 for x in lista]
print(f"Python zerrenda: {time.time() - hasiera:.3f}s")

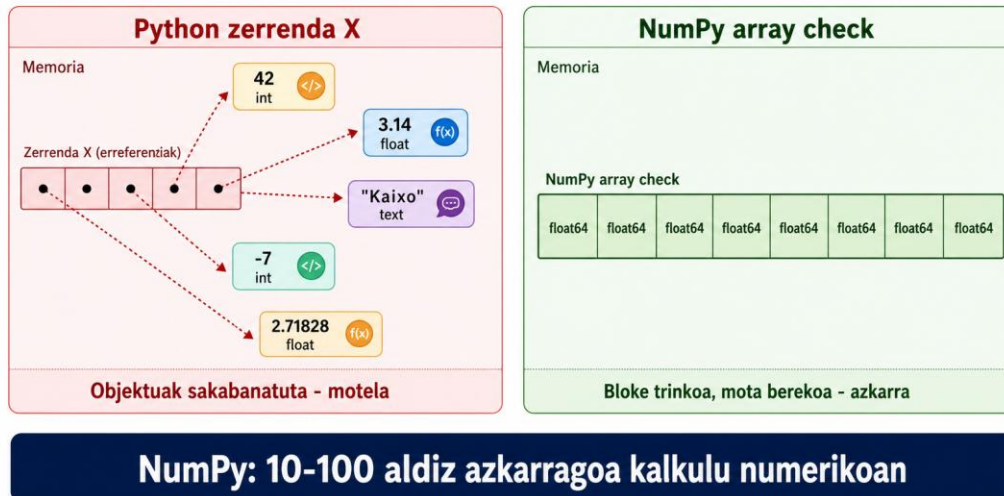
# NumPy array
array = np.arange(N)
hasiera = time.time()
emaitza = array * 2
print(f"NumPy array: {time.time() - hasiera:.3f}s")

# NumPy normalean 10-100 aldiz azkarragoa da
```

Lau dira NumPy azkarragoa izatearen arrazoiak: (1) **mota homoginoa** (elementu guztiak berdinak); (2) **C lengoian implementatua** dagoela (Python baino askoz azkarragoa); (3) **eragiketa bektorialak** (eragiketa array osoari aplikatzen zaio aldi berean, ez elementuz elementu); eta (4) **BLAS/LAPACK** liburutegi optimizatuak erabiltzen dituela algebra linealean.

NumPy instalatzeko, `pip install numpy` nahikoa da (Anaconda banaketak aurrez instalatuta dakar).

Python zerrenda vs NumPy array



1. Irudia: Python zerrendaren (objektu sakabanatuak memorian) eta NumPy arrayaren (bloke trinkoa) arteko aldea, abiadura-konparaketarekin.

Q Adibidea: Sentsore-datuak fabrika Fabrika batean, 200 sentsorek tenperatura neurtzen dute segundoko. Egun bakar batean 17 milioi neurketa pilatzen dira. Datu horiek Python zerrenda batekin prozesatuz gero (batezbestekoa, gehienezkoa, alarmak), kalkuluek minutuak hartuko lituzkete. NumPy array batekin, segundo gutxitan egiten da. Hori da NumPy fabrikako monitorizazio-sisteman ezinbestekoa izatearen arrazoia: datuak iristen diren erritmoan prozesatu behar dira.

! Ideia nagusia NumPy ez duzu beti zuzenean ikusten, baina Pandas, scikit-learn eta sare neuronaletan azpian dago beti. Bere indarra mota homogeneoan eta C-z idatzitako eragiketa bektorialetan datza: horregatik da 10-100 aldiz azkarragoa Python zerrendak baino.

⚠ Kontuz: array-ak ez dira zerrenda flexibleak NumPy array batean ezin dira mota desberdineko elementuak nahastu (zenbakiak eta kateak batera, adibidez). Eta tamaina aldatzea (elementuak gehitzea banan-banan) garestia da, zerrendekin ez bezala. Array-ak datu numeriko homogeneo eta tamaina egonkorrentzat dira; mota nahasiak edo etengabe hazten diren bildumak behar badituzu, Python zerrenda edo Pandas erabili.

□ Ariketa 1.1 • 15 min • banaka Egoera: Goiko denbora-konparaketaren scripta exekutatu behar duzu. **Eginkizuna:** Ingurune birtual batean `numpy` instalatu, kopia goiko kodea, eta exekutatu $N = 1_000_000$, $N = 10_000_000$ eta $N = 50_000_000$ baliotarako. Idatzi taula txiki bat lortutako denborekin. **Eztabaidatzeko:** Zenbat aldiz azkarragoa da NumPy kasu bakoitzean? Zergatik handitzen da aldea N handitzen denean?

1.2 Array-ak Sortu eta Manipulatu

Array bat NumPy-ren oinarritzko datu-egitura da: tamaina finkoko eta mota homogeneousko elementu-bilduma bat, edozein dimentsiotakoa izan daitekeena. Bektore bat (1D), matrize bat (2D) edo tentsor bat (3D edo gehiago) izan daiteke. Atal honetan array-ak nola sortu (zerrenden bidez, sortzaile-funtzioen bidez edo ausazko balioekin) eta haien forma nola aldatu ikasiko dugu.

Hasteko, modurik intuitiboena Python-eko zerrendetatik abiatzea da. Hortik aurrera, array bakoitzak hiru ezaugarri garrantzitsu ditu: `shape` (neurriak adierazten dituen tupla), `ndim` (dimentsio kopurua) eta `size` (elementu kopuru osoa). Ezaugarri hauek ezinbestekoak dira kodea garatzean, formekin egindako akatsak baitira errore-iturri ohikoenetako batzuk.

```
import numpy as np

# Zerrendatik array sortu
a = np.array([1, 2, 3, 4, 5])
print(a.dtype)    # int64

# Matrize bidimentsionala (2D)
matrizea = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])
print(matrizea.shape)    # (3, 3) - 3 errenkada, 3 zutabe
print(matrizea.ndim)     # 2 - dimentsio kopurua
print(matrizea.size)     # 9 - elementu kopuru osoa
```

Askotan ez dugu eskuz idatziko array baten edukia; aitzitik, forma jakin batekin sortu nahi izango dugu: zeroz beteta, batekoekin, balio jakin batez, edo ausaz. NumPy-k funtzio bereziak eskaintzen ditu egoera horietarako. ML proiektuetan bereziki ohikoak dira ausazko array-ak, pisuak hasieratzeko edo datu sintetikoak sortzeko.

```
import numpy as np

zeros = np.zeros((3, 4))    # 3x4 matrizea, denok 0
ones = np.ones((2, 3))     # 2x3 matrizea, denok 1
beteak = np.full((3, 3), 7.5) # 3x3 matrizea, denok 7.5
I = np.eye(4)               # 4x4 identitate-matrizea (diagonalean 1)

a = np.arange(0, 10, 2)     # [0 2 4 6 8] - 0tik 10era, 2ko pausuak
b = np.linspace(0, 1, 5)    # [0. 0.25 0.5 0.75 1.] - 5 balio berdin banatuta

# Ausazko array-ak
np.random.seed(42)          # Erreproduzigarritasunerako
c = np.random.rand(3, 3)    # [0, 1) tarteko balioak
d = np.random.randn(3, 3)   # Banaketa normala (μ=0, σ=1)
```

`np.random.seed()` deia oso garrantzitsua da: ausazko zenbakien sortzailearen "hazia" finkatzen du, eta horri esker emaitzak erreproduzigarriak izango dira exekuzio batetik bestera. Pentsa ezazu errezeta

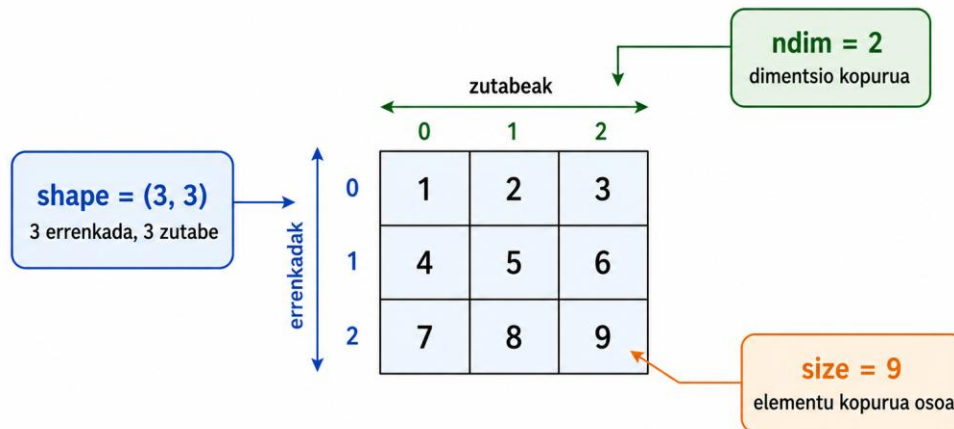
bat bezala: hazi bera erabiliz gero, beti ausazko zenbaki-segida bera lortuko duzu. Hau ezinbestekoa da esperimentuetan eta arazketan, akatsa berriz sortzeko gai izan behar baituzu.

Azkenik, **forma aldatzeko** aukera dago (**reshape**), datuak fisikoki mugitu gabe: 12 elementuko bektore bat 3×4 matrize bihurtu daiteke momentu batean. **-1** balioak NumPy-ri dimentsio bat automatikoki kalkulatzeko esaten dio.

```
import numpy as np

a = np.arange(12)          # [0 1 2 ... 11]
b = a.reshape(3, 4)        # 3 errenkada, 4 zutabe
c = a.reshape(4, -1)       # 4 errenkada, zutabe-kopurua automatikoa (3)
laua = b.flatten()         # Berriz bektore bakar bihurtu
```

Array baten anatomia



2. Irudia: 3×3 matrize baten anatomia bisuala — shape (3,3), ndim (2) eta size (9) etiketatuta, errenkada eta zutabeen norabidearekin.

Q Adibidea: Irudi bat array bihurtuta Argazki-aplikazio batean, koloretako irudi bat (640×480 pixel) NumPy array gisa irudikatzen da: forma (480, 640, 3) du (altuera, zabalera, RGB hiru kanalak). Iragazki bat aplikatzeko (adibidez, distira igotzeko), array osoari eragiketa bat aplikatzen zaio. Eta sare neuronal batean sartu aurretik, **reshape(-1)**-ekin pixel guztiak bektore bakar batean "lantu" daitezke. Irudia = array; horregatik da NumPy ezinbestekoa ikusmen artifizialean.

! Ideia nagusia Array bat sortzeko hainbat bide daude: **np.array** (zerrendatik), **np.zeros/np.ones/np.full** (forma jakin batekin), **np.arange/np.linspace** (sekuentziak) eta **np.random** (ausazkoak). **shape**, **ndim** eta **size** beti gogoan: forma-akatsak dira hasiberrien errore ohikoena.

□ **Ariketa 1.2 · 20 min · banaka Egoera:** Klase bateko 6 ikasleren 4 azterketako notak dituzu.

Eginkizuna:

1. Sortu 6×4 array bat ausazko notekin (`np.random.randint(0, 11, (6, 4))`), hazia 42-an finkatuta.
2. Inprimatu bere `shape`, `ndim` eta `size`.
3. `reshape`-ekin bihurtu 4×6 array batean, eta gero `flatten`-ekin bektore bakar batean.
4. Azaldu: zergatik funtzionatzen du `reshape(4, 6)`-ek baina ez `reshape(5, 5)`-ek?

1.3 Eragiketa Bektorialak eta Broadcasting

NumPy-ren benetako indarra eragiketa bektorialetan datza: aritmetika eta funtzio matematikoak array osoari aplikatzeko aukera, elementuz elementuko begizta bat idatzi gabe. Honek ez ditu soilik kode laburragoak eta irakurterazagoak ematen, errendimendu hobe ere bai. Atal honetan oinarriko eragiketak eta **broadcasting** mekanismoa landuko ditugu.

NumPy-n eragiketa aritmetikoak elementuz elementu aplikatzen dira, `for`-begiztarik gabe. Bi array batuz gero (`a + b`), posizio bakoitzeko balioak batzen dira automatikoki. Eta eskalar batekin (zenbaki bakar bat) eragiten baduzu (`a * 3`), array osoari aplikatzen zaio.

```
import numpy as np

a = np.array([1, 2, 3, 4, 5])
b = np.array([10, 20, 30, 40, 50])

print(a + b)          # [11 22 33 44 55]
print(a * b)          # [ 10  40  90 160 250]
print(a ** 2)         # [ 1  4  9 16 25]
print(np.sqrt(a))     # [1.   1.414 1.732 2.   2.236]
print(a * 3)          # [ 3  6  9 12 15] - eskalarra array osoari
```

Broadcasting NumPy-ren ezaugarri kontzeptualki interesgarrienetako bat da. Forma desberdineko array-en artean eragiketak zuzenean egiteko aukera ematen du: NumPy-k array txikiagoa "hedatzen" du handiagora egokitzeko, memoria gehigarririk gabe. Pentsa ezazu Excel-en: zutabe bati balio bat batzen badiozu, balio hori errenkada guztietan aplikatzen da automatikoki. Broadcasting kontzeptu beraren bertsio orokortua da.

```
import numpy as np

# 2D array + 1D array: errenkada "hedatu" egiten da
matrizea = np.array([[1, 2, 3],
                     [4, 5, 6],
                     [7, 8, 9]])
errenkada = np.array([10, 20, 30])

print(matrizea + errenkada)
# [[11 22 33]
#   [14 25 36]
#   [17 28 39]]
```

```
# errenkada array-a 3 errenkadatan "hedatzen" da automatikoki
```

Broadcasting-aren araua sinplea da: formak eskuinetik alderatzen dira; dimentsio bat 1 bada (edo falta bada), beste array-aren tamainara hedatzen da; tamainak bateragarriak ez badira, errorea gertatzen da. Adibide praktiko ezin hobea **datuen normalizazioa** (Z-score) da, ML-ren aurreprozesamendu-pauso ohikoenetako bat:

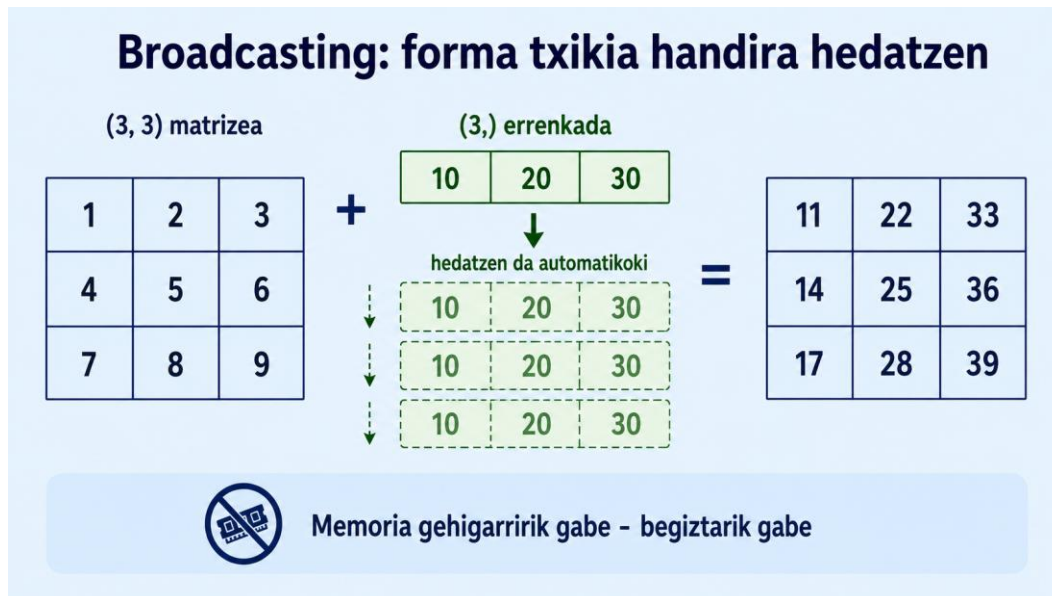
```
import numpy as np

datuak = np.array([
    [2.0, 3.0, 4.0],
    [5.0, 6.0, 7.0],
    [8.0, 9.0, 10.0]
])

# Zutabe bakoitzaren batezbestekoa eta desbideratze estandarra
batezbestekoa = datuak.mean(axis=0) # forma: (3,)
std = datuak.std(axis=0)             # forma: (3,)

# Broadcasting bidez normalizatu (Z-score) - lerro bakarra!
normalizatua = (datuak - batezbestekoa) / std
```

Eskuz eginez gero, hainbat begizta erabili beharko lirateke; NumPy-rekin, lerro bakarra nahikoa da. Eta askoz azkarrago exekutatzen da.



3. Irudia: Broadcasting-aren mekanismoa bisualki — (3,) formako errenkada-array bat (3,3) matrize batera nola "hedatzen" den, eragiketa elementuz elementu aplikatuz.

Q Adibidea: Prezioei BEZa gehitzea dendako sisteman Online denda batek 5.000 produkturen prezioak ditu NumPy array batean. BEZ %21 gehitu behar zaie guztiei salmenta-prezioa kalkulatzeko. Begizta batekin: *for* bat 5.000 iterazioarekin. Broadcasting-

ekin: `salmenta_prezioak = prezioak * 1.21`. Lerro bakarra, eta gainera kategorია bakoitzeko BEZ desberdina aplikatu nahi bada (liburuak %4, gainerakoak %21), kategoria-bektore bat ere broadcasting bidez aplikatu daiteke. Eskuzko begiztarik gabe.

! Ideia nagusia Eragiketa bektorialek eta broadcasting-ek elkar osatzen dute: kode kontzizu, irakurterraz eta eraginkorra idazteko aukera ematen dute. "For begizta bat idazten ari bazara NumPy array baten gainean, ziurrenik modu bektorial bat existitzen da, azkarragoa eta argiagoa".

⚠ Kontuz: forma-bateragarritasuna Broadcasting-ak baldintza batzuk eskatzen ditu. `(3,3)` matrize bati `(3,)` bektore bat batu diezaiokezu, baina ez `(4,)` bektore bat: `ValueError: operands could not be broadcast together` errorea jasoko duzu. Errore hau oso ohikoa da. Akatsa topatzeko, beti inprimatu array bakoitzaren `.shape` eragiketaren aurretik.

□ **Ariketa 1.3 · 20 min · taldeka Egoera:** Denda batek 4 produkturen oinarritzko prezioak ditu: `[10.0, 25.0, 8.0, 50.0]`. **Eginkizuna** (broadcasting erabiliz, begiztarik gabe): 1. Gehitu BEZ %21 produktu guztiei. 2. Aplikatu %10eko deskontua bigarren eta laugarren produktuei soilik (pista: deskontu-bektore bat sortu, `[1.0, 0.9, 1.0, 0.9]`). 3. Datuen normalizazioa: sortu `5x4` matrize bat (5 denda, 4 produktu) eta normalizatu zutabe bakoitza Z-score-arekin. **Eztabaidatzeko:** Zer abantaila du honek `for` begizta batekin alderatuta?

1.4 Indexazioa eta Zatiketa

Behin array-ak sortu eta horiekin eragiketak egiten ikasita, hurrengo urratsa elementu zehatzak nola atzitu, hautatu edo aldatu ikastea da. NumPy-k hainbat estrategia eskaintzen ditu: indizeen bidezko sarbide klasikoa, zatiketa (slicing), eta maska logikoen bidezko hautaketa. **Boolean indexazioa**, bereziki, datu-zientzian etengabe erabiltzen den teknika da.

Oinarritzko indexazioa Python zerrenden oso antzekoa da: indizeak 0-tik hasten dira, indize negatiboak amaieratik kontatzen dira (`a[-1]` azkena da), eta zatiketak `[hasiera:amaiera:pausua]` formatua jarraitzen du. Bi dimentsioko array-etan, errenkada eta zutabea aldi berean adierazten dira komaz bereizita: `m[errenkada, zutabea]`.

```
import numpy as np

m = np.array([
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12]
])

print(m[0, 0])      # 1 - lehen errenkada, lehen zutabea
print(m[1, 2])      # 7
print(m[1, :])      # [5 6 7 8] - errenkada osoa
print(m[:, 2])      # [3 7 11] - zutabe osoa
```

```
print(m[0:2, 1:3]) # azpi-matrizea: [[2 3] [6 7]]
```

Boolean indexazioa (maskak) datu-zientziaren eguneroko lanaren ardatzetako bat da. Baldintza bat aplikatzean ($a > 10$) Boolean array bat sortzen da (maska), eta horren bidez baldintza betetzen duten elementuak hautatu daitezke. Hau askoz garbiagoa eta azkarragoa da begizta esplizituetan baino. Hainbat baldintza konbinatzeko $\&$ (eta), $|$ (edo) eta $\sim$ (ez) erabiltzen dira, gogoratu **parentesiak derrigorrezkoak** direla baldintza bakoitzaren inguruan.

```
import numpy as np

a = np.array([15, 3, 8, 22, 7, 19, 4, 11])

maska = a > 10

print(a[maska])      # [15 22 19 11]
print(a[a > 10])     # Berdina, zuzenean

# Hainbat baldintza (parentesiak derrigorrezkoak!)
print(a[(a > 5) & (a < 15)]) # [ 8  7 11]

# Balioak aldatu maskaren bidez
a[a > 15] = 0
print(a) # [ 0  3  8  0  7  0  4 11]

# NaN balioak tratatu
datuak = np.array([1.0, np.nan, 3.0, np.nan, 5.0])
baliodunak = ~np.isnan(datuak) # ~ = EZ logikoa
print(np.mean(datuak[baliiodunak])) # 3.0 - NaN-ak baztertuta
```

Boolean maska: baldintza -> iragazketa



4. Irudia: Boolean indexazioaren prozesua — jatorrizko array bat, baldintza batek sortutako True/False maska, eta emaitzako array iragazia.

Q Adibidea: Bezero-segmentazioa banketxean Banketxe batek bezeroen urteko sarrerak ditu NumPy array batean. Marketin-kanpaina batek 30.000€tik gorako eta 60.000€tik beherako bezeroei zuzendu nahi die (klase ertaina). Maska bakar batekin: $\text{bezeroak}[(\text{sarrerak} > 30000) \& (\text{sarrerak} < 60000)]$. Milioika bezeroen artean

baldintza betetzen dutenak segundo gutxitan iragazten dira, begiztarik gabe. Boolean indexazioa = enpresaren datu-segmentazioaren oinarria.

💡 **Idea nagusia** Boolean maskak NumPy-ko tresnarik indartsuenetako batzuk dira: baldintza bat array bati aplikatu, True/False maska bat lortu, eta horrekin datuak iragazi edo aldatu. Hainbat baldintza konbinatzean, gogoratu beti parentesiak: $(a > 5) \ \& \ (a < 15)$, ez $a > 5 \ \& \ a < 15$.

⚠ **Kontuz:** *and/or ez, &/| bai*

NumPy maskekin **ezin dira** Python-en *and* eta *or* erabili (errorea jaurtitzen dute array-ekin). Beti *&* (eta) eta *|* (edo) erabili behar dira, parentesiekin. Akats hau oso ohikoa da Python-etik datozenentzat.

□ **Ariketa 1.4 · 15 min · banaka Egoera:** Saltoki bateko 10 produkturen stock-a duzu: `np.array([12, 0, 45, 3, 0, 78, 5, 0, 23, 1])`. **Eginkizuna** (boolean indexazioa erabiliz): 1. Aurkitu zenbat produktu dauden agortuta (stock = 0). 2. Lortu stock baxuko produktuak (0 baino handiago baina 5 baino txikiago). 3. Stock baxuko produktu guztiei (stock < 5) balioa 10ra aldatu (birhornitzea simulatuz).

1.5 Estatistika eta Algebra Linealaren Oinarriak

NumPy-k estatistika deskribatzaileko eta algebra linealeko funtzio aberatsen multzoa eskaintzen du. Hauek ez dira soilik teoriarako tresnak, baizik eta erregresioa, dimentsio-murrizketa edo sare neuronaletako pisuen kalkulurako oinarri praktikoak ere bai. Lasai, hemen ez dugu deribazio matematikorik egingo; funtzioak nola erabili eta zertarako balio duten ikusiko dugu, intuizio praktikoarekin.

Estatistika deskribatzaileko funtzioak zuzenak dira: *mean* (batezbestekoa), *median* (mediana), *std* (desbideratze estandarra), *min*, *max*, *sum*, eta abar. Matrizeetan, **axis parametroa** funtsezkoa da: *axis=0*-k zutabeen norabidean eragiten du (errenkadak agregatuz), eta *axis=1*-ek errenkaden norabidean (zutabeak agregatuz). Modu praktikoa hau ulertzeko: "zein ardatz desagertzen den emaitzan".

```
import numpy as np

datuak = np.array([4.0, 7.0, 13.0, 2.0, 1.0, 9.0, 6.0, 3.0, 8.0, 5.0])

print(f"Batezbestekoa:      {np.mean(datuak):.2f}")      # 5.80
print(f"Mediana:           {np.median(datuak):.2f}")     # 5.50
print(f"Desbideratze est.: {np.std(datuak):.2f}")        # 3.19
print(f"Pertzentila 25:    {np.percentile(datuak, 25):.2f}") # 3.25

# Matrizeetan: axis parametroa
m = np.array([[1, 2, 3],
               [4, 5, 6],
               [7, 8, 9]])
```

```
print(m.mean(axis=0)) # [4. 5. 6.] - zutabe bakoitzaren batezbestekoa
print(m.mean(axis=1)) # [2. 5. 8.] - errenkada bakoitzaren batezbestekoa
```

Algebra lineala datu-zientziaren oinarri matematikoa da: erregresio linealak, dimentsio-murrizketak (PCA) eta sare neuronalen propagazioa, denak matrize-eragiketetan oinarritzen dira. Hemen intuizioa garrantzitsua da: matrize bat datu-taula bat da, eta matrize-biderketa hainbat datu-multzo aldi berean konbinatzeko modua da. Garrantzitsua da bereiztea: $A * B$ elementuz elementuko biderketa da (posizio bakoitza bere parearekin), eta $A @ B$ (edo `np.dot`) benetako **matrize-biderketa** da (sare neuronaletan behin eta berriz erabiltzen dena).

```
import numpy as np

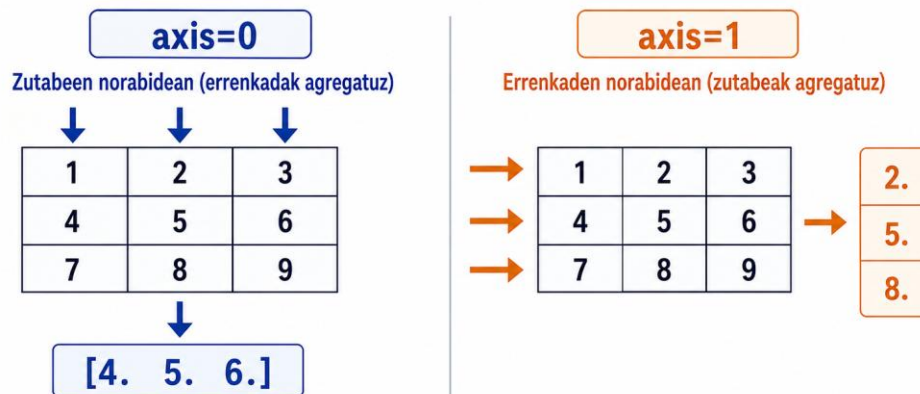
A = np.array([[1, 2],
              [3, 4]])

B = np.array([[5, 6],
              [7, 8]])

C = A @ B      # Matrize-biderketa (ez elementuz elementu!)
print(A.T)     # Transposazioa: errenkadak ↔ zutabeak

# linalg modulua: eragiketa aurreratuak
det = np.linalg.det(A)      # Determinantea
A_inv = np.linalg.inv(A)    # Alderantzizkoa
balioak, bektoreak = np.linalg.eig(A) # Balio/bektore propioak
```

axis=0 vs axis=1 - zein ardatza desagertzen den



Truko praktikoa: zein ardatza desagertzen da emaitzan?

5. Irudia: `axis` parametroaren intuizioa — $axis=0$ -k zutabeen norabidean agregatzen du (gora-behera geziak), $axis=1$ -ek errenkaden norabidean (ezker-eskuin geziak), bakoitzaren emaitzarekin.

Q Adibidea: Saltoki-kateko salmenten laburpena Saltoki-kate batek 5 denda $\times$ 12 hilabeteko salmenta-matrizea du ($(5, 12)$ array bat). `salmentak.sum(axis=1)`-ekin

denda bakoitzaren urteko salmenta osoa lortzen da (12 hilabeteak batuta, hilabete-ardatza desagertu). `salmentak.mean(axis=0)`-ekin, hilabete bakoitzeko batezbesteko salmenta dendetan zehar (denda-ardatza desagertu). `axis` ondo aukeratuz, txosten osoa lerro gutxitan eraikitzen da.

💡 **Idea nagusia** Estatistika deskribatzailerako, gakoa `axis` parametroa zuzen erabiltzea da: "zein ardatz desagertzen den emaitzan". Algebra linealean, gogoratu `*` (elementuz elementu) eta `@` (matrize-biderketa) ez direla gauza bera; ML-en ia beti `@` erabiltzen da.

⚠ **Kontuz:** `np.linalg.inv` eta **matrize singularrak** Matrize baten alderantzizkoa kalkulatzeko (`np.linalg.inv`), matrizeak ez-singularra izan behar du (determinantea ez zero). Matrize singular batekin saiaturaz gero, errorea sortzen da. Praktikan, ekuazio-sistemak ebazteko, sarritan `np.linalg.solve` gomendatzen da, egonkorragoa eta azkarragoa baita.

□ **Ariketa 1.5 · 20 min · taldeka Egoera:** Hiru dendaren lau hiruhilekoko salmenta-matrizea duzu (milaka eurotan): `np.array([[120, 135, 150, 165], [90, 95, 110, 100], [200, 210, 195, 220]])`. **Erregimenak:** 1. Kalkulatu denda bakoitzaren urteko salmenta osoa (`axis` egokia). 2. Kalkulatu hiruhileko bakoitzeko batezbesteko salmenta dendetan zehar. 3. Aurkitu zein dendak duen salmenta-aldakortasun handiena (`std` erabiliz errenkadetan). **Eztabaidatzeko:** Zein `axis` erabili duzu kasu bakoitzean, eta zergatik?

2. Pandas: Datu-taulen Kudeaketa

Pandas datu-zientziarekin lan egitean gehien erabiltzen den Python liburutegia da. NumPy-ren gainean eraikia dago, eta hari esker datu egituratuak (taulekin) lan egitea Excel edo SQL-rekin lan egitea bezain intuitiboa da, baina askoz aukera gehiagorekin. Atal honetan Series eta DataFrame egiturak, datuak kargatzeko moduak (CSV, Excel, JSON, SQL), datu-garbiketa, agregazioa eta taulen batzea ikasiko dugu. Trebetasun hauek ezinbestekoak dira edozein ML-proiekturen aurreprozesamendu-fasean, sarritan denbora gehien hartzen duen fasea izaten baita.

2.1 Series eta DataFrame

Pandas-ek bi datu-egitura nagusi ditu: **Series** (zutabe bakarra, indize etiketatuarekin) eta **DataFrame** (errenkadak eta zutabeak dituen taula, Excel orri baten antzekoa). Biak NumPy array-en gainean eraikiak daude, baina etiketak (indizeak eta zutabe-izenak) gehitzen dizkiete, datuekin hizkuntza naturalagoan lan egiteko aukera emanez. Pentsa ezazu Series bat Excel-eko zutabe bakar bat bezala, eta DataFrame bat Excel-eko orri oso bat bezala.

```
import pandas as pd

# Series: zutabe bakarra, indize etiketatuarekin
populazioa = pd.Series({
    "Bilbo": 346_843,
    "Donostia": 187_698,
    "Gasteiz": 252_427,
})
print(populazioa["Bilbo"])          # 346843
print(populazioa[populazioa > 200_000]) # Boolean indexazioa
```

DataFrame-a Pandas-en datu-egitura nagusia da: bi dimentsioko taula bat, zutabe bakoitza Series bat dela. ML-proiektuetan, datu-multzoak ia beti DataFrame gisa irudikatzen dira. Behin sortuta, hainbat metodo erabilgarri ditu datuak ezagutzeko: `shape` (neurriak), `dtypes` (zutabe-motak), `describe()` (estatistika-laburpena), `head()`/`tail()` (lehen/azken errenkadak).

```
import pandas as pd

datuak = {
    "izena": ["Ane", "Mikel", "Leire", "Jon", "Amaia"],
    "adina": [25, 30, 22, 35, 28],
    "hiria": ["Bilbo", "Donostia", "Gasteiz", "Bilbo", "Donostia"],
    "soldata": [28000, 35000, 24000, 42000, 31000]
}
df = pd.DataFrame(datuak)

print(df.shape)          # (5, 4) - 5 errenkada, 4 zutabe
print(df.describe())     # Estatistika-laburpena zenbakizko zutabeetan
print(df.head(3))        # Lehen 3 errenkada

# Zutabeak aukeratu
print(df["izena"])        # Series bat
print(df[["izena", "adina"]]) # DataFrame bat

# Errenkadak filtratu (baldintza baten arabera)
bilbotarrak = df[df["hiria"] == "Bilbo"]
hautaketa = df[(df["adina"] < 30) & (df["soldata"] > 25000)]
```

Errenkadak atzitzeko bi modu daude: `iloc` posizioaren arabera (da `df.iloc[0]` lehen errenkada) eta `loc` etiketaren arabera. Filtratze-eragiketak NumPy-ren boolean indexazioarekin oso antzekoak dira: berriz ere `&`, `|` eta parentesiak.

Series vs DataFrame

Series		DataFrame				
Zutabe bakarra, indize etiketatuarekin		Hainbat zutabe + errenkadak				
	biztanleria	izena	adina	hiria	soldata	
Bilbo	346843	0	Ane	28	Bilbo	2450
Donostia	187698	1	Jon	34	Donostia	2650
Gasteiz	252427	2	Maite	29	Gasteiz	2300
		3	Iñigo	41	Bilbo	3200
Excel-eko zutabe bat bezala		Excel-eko orri oso bat bezala				



DataFrame-ko zutabe bakoitza Series bat da

6. Irudia: Series (zutabe bakarra, indize etiketatuarekin) eta DataFrame (hainbat zutabe + errenkadak) egituren konparaketa bisuala, Excel-orri batekin analogia.

Q Adibidea: RRHH saileko langile-taula Giza Baliabideen sail batek langile guztien datuak DataFrame batean ditu: izena, saila, soldata, sarrera-data. Zuzendariak galdetzen du: "Zenbat langilek irabazten dute 30.000€tik gora IT sailean?". Lerro bakar batekin: `df[(df["saila"] == "IT") & (df["soldata"] > 30000)].shape[0]`. Excel-en iragazki batzuk eskuz ezarri beharko lirateke; Pandas-ekin, galdera kode bakar batean adierazten da eta script gisa errepika daiteke hilerok.

Idea nagusia Series = zutabe bakarra; DataFrame = taula osoa. Bere sintaxia NumPy-ren oso antzekoa da, baina etiketen abantailarekin: zutabeak izenez atzitzen dituzu (`df["soldata"]`), ez posizio numerikoz. `iloc` posiziorako, `loc` etiketarako.

Kontuz: `df["a"]` vs `df[["a"]]` Korxete bakarrak (`df["izena"]`) **Series** bat itzultzen du; korxete bikoitzak (`df[["izena"]]`) **DataFrame** bat. Aldea garrantzitsua da: metodo batzuek Series-en bakarrik funtzionatzen dute eta beste batzuek DataFrame-etan. Akatsa jasotzen baduzu, egiaztatu zenbat korxete erabili dituzun.

Ariketa 2.1 · 20 min · banaka Egoera: Goiko langile-taula (5 langile) erabiliz.

Eginkizuna: 1. Sortu DataFrame-a eta inprimatu `shape`, `dtypes` eta `describe()`. 2. Lortu Donostiako langileak. 3. Lortu 28 urtetik gorako eta 30.000€tik gorako langileak. 4. Sortu zutabe berri bat (`soldata_urtekoa = soldata * 14`) 14 ordainsariren arabera.

2.2 Datuak Kargatu Iturri Anizkoitzetatik

Errealeko proiektuetan datuak iturri askotatik etortzen dira: CSV fitxategiak, Excel kalkulu-orriak, JSON APIetatik, SQL datu-baseetatik. Pandas-ek funtzio espezializatuak eskaintzen ditu iturri

bakoitzerako, gehienek `read_` aurrizkia daramatela (`read_csv`, `read_excel`, `read_json`, `read_sql`). Atal honetan iturri ohikoenak ikusiko ditugu.

CSV (Comma-Separated Values) datuak partekatzeko formatu unibertsalena da bere sinpletasunagatik. `read_csv` funtzioa oso malgua da: bereizlea (`sep`), kodifikazioa (`encoding`), balio faltan adierazpenak (`na_values`), datu-motak, eta abar. Garrantzitsua da `encoding` ondo zehaztea, batez ere euskaraz edo gaztelaniaz, azentu eta karaktere bereziekin.

```
import pandas as pd

# CSV irakurri parametroekin
df = pd.read_csv(
    "datuak.csv",
    sep=";",
    encoding="utf-8",
    na_values=["N/A", "?", "-"],
    parse_dates=["data"],
)

# CSV idatzi (index=False ohikoa da)
df.to_csv("emaitza.csv", index=False, sep=";", encoding="utf-8")
```

Excel fitxategiekin lan egitea sarritan beharrezkoa da enpresa-ingurunetan (`pip install openpyxl` behar da `.xlsx`-rako). Orri zehatza aukera daiteke (`sheet_name`), eta `ExcelWriter`-ekin orri anitz idatz daitezke fitxategi bakarrean. **JSON** APIetan ohikoa da (`read_json`, edo `json_normalize` egitura barneratuak lautzeko). **SQL** datu-baseak (`read_sql_query`) ekoizpen-inguruneetan oso ohikoak dira.

```
import pandas as pd

# Excel irakurri orri jakin batetik
df = pd.read_excel("datuak.xlsx", sheet_name="Txostena")

# JSON irakurri
df = pd.read_json("datuak.json")

# SQL kontsulta zuzenean DataFrame-ra
import sqlite3
konexioa = sqlite3.connect("datu_basea.db")
df = pd.read_sql_query("SELECT * FROM bezeroak WHERE adina > 25", conexioa)
konexioa.close()
```

Errealitatean, datu-multzo bera fitxategi askotan banatuta egoten da (adibidez, hilabeteko salmenta-erregistroak). `glob` modulua eta `concat` funtzioa konbinatuz, karpetako CSV guztiak modu automatizatuan kargatu eta batu daitezke. Ohitura ona da iturria adierazten duen zutabe bat gehitzea.

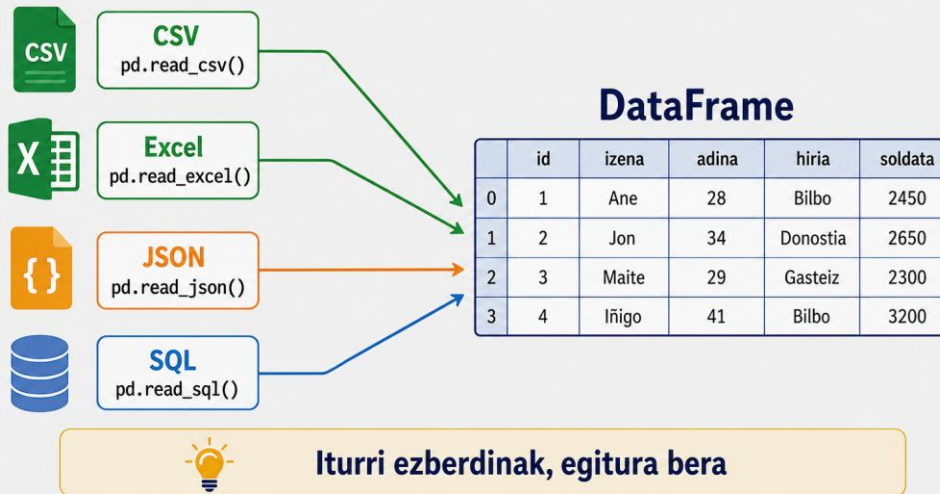
```
import pandas as pd
import glob

fitxategiak = glob.glob("datuak/*.csv")
zerrenda = []
for fitxategia in fitxategiak:
```

```
df_bat = pd.read_csv(fitxategia)
df_bat["iturria"] = fitxategia      # Nondik datorren markatu
zerrenda.append(df_bat)
```

```
df_osoa = pd.concat(zerrenda, ignore_index=True)
```

Pandas: iturri guztiak DataFrame batera



7. Irudia: Pandas-en datu-iturriak — CSV, Excel, JSON eta SQL ikonoak DataFrame zentral batera doazen geziekin, read_funtzioen izenak etiketatuta.

Q Adibidea: Hilabeteko salmenten bateratzea Denda-kate batek 12 CSV fitxategi sortzen ditu urtean, hilabeteko bat. Urtearen amaieran, txosten oso bat behar da. Eskuz: 12 fitxategi Excel-en ireki eta kopiatu-itsatsi, akats-iturri handia. Pandas-ekin: *glob* + *concat*-ek lerro gutxitan 12 fitxategiak DataFrame bakar batean batzen ditu, *iturria* zutabearekin hilabete bakoitza identifikatuta. Datu berriak iristen direnean, script bera errepikatzen da.

💡 Ideia nagusia Pandas-ek edozein datu-iturritatik kargatzen du *read_** funtzioekin. Gakoa parametro egokiak erabiltzea da: bereizlea, kodifikazioa eta datu-motak ondo zehazteak akats askotatik salbatuko zaitu. *encoding="utf-8"* ia beti behar duzu euskaraz lan egiteko.

⚠ Kontuz: kodifikazio-akatsak euskaraz CSV bat irakurtzean karaktere arraroak agertzen badira (Ã±, Ã¡...), kodifikazio okerra da seinale. Probatu *encoding="utf-8"* edo *encoding="latin-1"*. Windows-eko Excel-etik datozen fitxategiek askotan *encoding="cp1252"* edo *encoding="utf-8-sig"* behar dute. Ez baztertu hau "detaile txikia" gisa: akats hau etengabe agertzen da praktikan.

□ **Ariketa 2.2 · 25 min · taldeka Egoera:** Tutoreak hiru CSV fitxategi emango dizkizu (*urtarrila.csv*, *otsaila.csv*, *martxoa.csv*), salmenta-erregistroekin (data, produktua,

kopurua, prezioa). **Eginkizuna:** 1. Kargatu fitxategi bakoitza `read_csv`-rekin, `encoding` egokia erabiliz. 2. Gehitu `iturria` izeneko zutabe bat (hilabetearen izena) bakoitzari. 3. Batu hirurak `concat`-ekin DataFrame bakar batean. 4. Inprimatu zenbat errenkada dituen guztira eta hilabete bakoitzeko.

02.8 2.3 Datu-garbiketa: fillna, dropna eta beste

□ **Errepasoa: 5072 moduluan ikusi duzue** Datu-garbiketaren oinarri kontzeptualak (EDA, balio falten azterketa, MCAR/MAR/MNAR, outlier-en detekzioa, multikolinealtasuna, VIF) **5072 moduluan** (`5072_1_Datua_eta_Aurreprozesamenua`) landu dira sakon. Hemen Pandas API-rekin **praktikan jartzen** ikasiko dugu: `isnull`, `fillna`, `dropna`, `interpolate` — sintaxia eta eguneroko lan-fluxua.

Datu errealetan akatsak, balio falta eta inkoherentziak ohikoak dira. **Datu-garbiketa** (data cleaning) edozein ML proiektutan denbora gehien hartzen duen fasea da: sarritan proiektuaren %60-80. Hau ez da denbora-galera, baizik eta zuzeneko inbertsioa eredu fidagarriagoak lortzeko. Printzipioa argia da: "garbage in, garbage out" — datu zikinekin, prozesatze-teknika onenak ere alferrikakoak dira.

! **Agente-erabilera ona** (gogoratu 1. gaiko §2.5 ohiturak) Datu-garbiketa Pandas-en errepikakorra da, eta hor agentea ondo erabiltzen jakiteak aldea egiten du. Eskatu agenteari kontestutarekin: "Hona nire CSV-aren lehenengo 5 errenkadak (`df.head().to_dict()`), eta zutabeen `dtypes`. Idatzi `garbitu(df)` funtzio bat hutsegite-balioak betetzen dituen (`age` mediana, `country` 'unknown'), outlier-ak `score` zutabeen kentzen dituen (`3σ` atalasea) eta `dtypes` zutabe bakoitzari dagokion motara bihurtzen dituen. Itzuli funtzioa + `pytest` test bat hutsegitearen kasurako." Iteratzen joan, ez idatzi mega-prompt bat hasieran (§2.5.4).

Ezer baino lehen, **balio falten azterketa** egiten da. `isnull().sum()`-ek zutabe bakoitzeko balio falten kopurua ematen du; ehunekoa kalkulatzuz, erabaki daiteke zer egin: zutabe batean balio falten ehunekoa oso altua bada (adibidez, %70), agian hobe da zutabe hori baztertzea; ehuneko txikiak inputazioz tratatu daitezke.

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "izena":    ["Ane", "Mikel", None, "Jon", "Leire"],
    "adina":    [25, np.nan, 30, np.nan, 28],
    "soldata":  [28000, 35000, np.nan, 42000, 31000],
})

print(df.isnull().sum())           # Balio falten kopurua zutabeka
print(df.isnull().sum() / len(df) * 100) # Ehunekoa
print(df[df.isnull().any(axis=1)])    # Balio faltadun errenkadak
```

Bi estrategia nagusi daude: **dropna** (balio faltadun errenkadak/zutabeak ezabatu) eta **fillna** (balioak bete). **dropna** egokia da datu-galera txikia denean; kontuz, ordea, errenkadak modu kritikorik gabe kentzeak datu-multzoa asko murriztu baitezake. **fillna**-k betetzeko hainbat aukera ditu: balio finkoa, estatistika bat (batezbestekoa, mediana), aurreko/ondorengo balioa (**ffill**/**bfill**, denbora-serieetan ohikoa) edo interpolazioa.

```
import pandas as pd
import numpy as np

# dropna: balio faltadun errenkadak ezabatu
df_garbia = df.dropna()
df_garbia = df.dropna(subset=["adina", "soldata"]) # Zutabe zehatzetan
df_garbia = df.dropna(thresh=3) # Gutxienez 3 balio baliodun behar ditu

# fillna: balioak bete
df_beteta = df.fillna({
    "adina": df["adina"].mean(), # Batezbestekoarekin
    "soldata": 0, # Zeroarekin
    "izena": "Ezezaguna", # Katearekin
})

# Denbora-serieetan: aurreko balioarekin edo interpolazioz
tenp = pd.Series([20.0, np.nan, 22.0, np.nan, 19.0])
tenp.fffll() # 20.0, 20.0, 22.0, 22.0, 19.0
tenp.interpolate() # 20.0, 21.0, 22.0, 20.5, 19.0 - tartekatze lineala
```

Garbiketak beste bi atal nagusi ditu: **bikoiztuak ezabatzea** (**drop_duplicates**) eta **mota-bihurketa eta normalizazioa**. Datuak fitxategitik datozenean, sarritan kate (string) gisa interpretatzen dira, nahiz eta zenbaki edo data izan beharko luketen. **astype**, **pd.to_datetime** eta string-metodoek (**.str.replace**, **.str.strip**) datuak normalizatzeko aukera ematen dute.

```
import pandas as pd

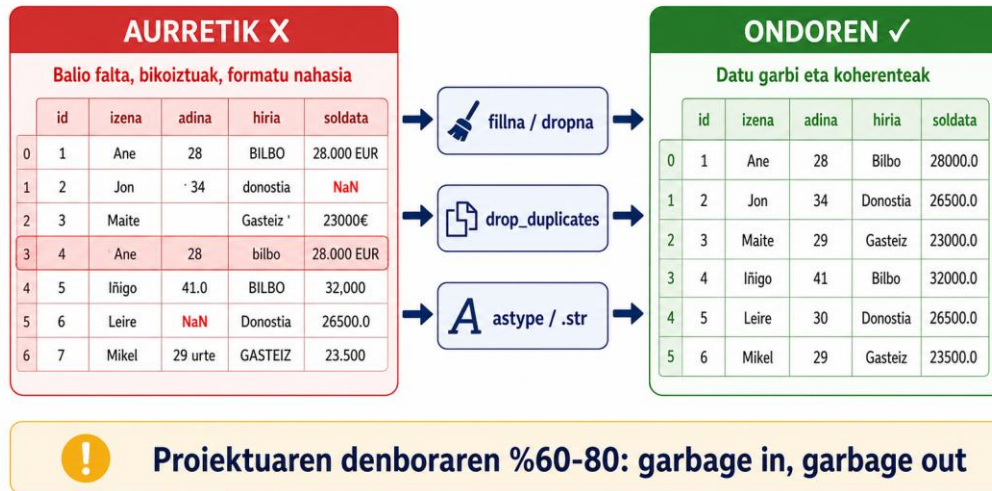
df = pd.DataFrame({
    "soldata": ["28.000€", "35.000€", "24.000€"],
    "data": ["2024-01-15", "2024-02-20", "2024-03-10"],
    "izena": ["Ane ", "MIKEL", "leire"]
})

# Bikoiztuak kendu
df = df.drop_duplicates(subset=["izena"], keep="first")

# Kate garbitu eta zenbakira bihurtu
df["soldata"] = df["soldata"].str.replace("€", "").str.replace(".", "").astype(float)

# Data-formatua eta kate-garbiketa
df["data"] = pd.to_datetime(df["data"])
df["izena"] = df["izena"].str.strip().str.capitalize()
```


Datu-garbiketa: zikinetik garbira



8. Irudia: Datu zikinetatik datu garbietara — balio falten tratamendua, bikoiztuen ezabaketa eta mota-normalizazioa, aurretik/ondoren taula batekin.

Q Adibidea: CRM bateko bezero-datu zikinak Enpresa batek bezero-datuak hainbat iturritatik bildu ditu (web-formularioak, telefono-deiak, inportazioak). Emaiza: izen batzuk maiuskulaz, beste batzuk minuskulaz; telefono batzuk gidoiekin, beste batzuk gabe; e-postak hutsik. Marketin-kanpainaren aurretik, garbitu egin behar da:

`.str.strip().str.lower()` izenetan, e-posta hutsak `dropna`-rekin baztertu, bikoiztuak `drop_duplicates`-ekin kendu. Datu garbiekin, kanpaina askoz eraginkorragoa da: ez da e-postarik bikoizten ezta okertzen.

! Ideia nagusia Datu-garbiketa proiektuaren denboraren %60-80 da, eta hori normala da: "garbage in, garbage out". Balio faltentzat, lehenik aztertu (`isnull().sum()`), gero erabaki: ezabatu (`dropna`) ala bete (`fillna`). Erabakia testuinguruak agintzen du, ez automatismoak.

□ **Hausnartzeko** Imaginatu 1.000 errenkadako datu-multzo batean, adin-zutabean %3 dago hutsik. Bi aukera: errenkada horiek ezabatu (`dropna`) ala batezbestekoarekin bete (`fillna`). Zein aukeratuko zenuke? Eta hutsa %40 balitz? Zer gertatzen da batezbestekoarekin betetzen baduzu hutsune asko daudenean?

□ **Ariketa 2.3 · 25 min · taldeka Egoera:** Tutoreak datu zikinak dituen CSV bat emango dizu (bezeroak: izena, adina, hiria, soldata), balio faltarekin, bikoiztuekin eta formatu-akatsekin. **Eginkizuna:** 1. Aztertu balio falten ehunekoa zutabeka. 2. Bete adina batezbestekoarekin eta hiria "Ezezaguna"-rekin. 3. Kendu bezero bikoiztuak (izenaren arabera). 4. Garbitu izenak (zuriuneak, maiuskulak) eta soldata zenbaki bihurtu. 5. Konparatu hasierako eta amaierako errenkada-kopurua.

2.4 Agregazioa: groupby eta pivot

Datu-agregazioa datu-analisi esploratzailearen ardatza da: datu xehetuetatik (errenkada bakoitza erregistro indibidual bat) ikuspegi laburtuak lortzeko aukera ematen du. SQL-en **GROUP BY** eta Excel-en taula dinamikoen baliokideak Pandas-en **groupby** eta **pivot_table** funtzioak dira.

groupby funtzioak **"split-apply-combine"** eredu klasikoa inplementatzen du: datuak gakoaren arabera zatitzen dira, eragiketa bat aplikatzen zaio zati bakoitzari, eta emaitzak elkartzen dira. **agg** metodoaren bidez hainbat agregazio aldi berean defini daitezke, zutabe-izen pertsonalizatuekin.

```
import pandas as pd

df = pd.DataFrame({
    "departamentua": ["IT", "HR", "IT", "HR", "IT", "Finantza"],
    "hiria":          ["Bilbo", "Bilbo", "Donostia", "Bilbo", "Donostia", "Gasteiz"],
    "soldata":        [42000, 30000, 38000, 28000, 45000, 52000],
})

# Departamentuka batezbesteko soldata
print(df.groupby("departamentua")["soldata"].mean())

# Hainbat agregazio zutabe-izen pertsonalizatuekin
laburpena = df.groupby("departamentua").agg(
    soldata_batezb=("soldata", "mean"),
    soldata_max=("soldata", "max"),
    langile_kop=("soldata", "count"),
).round(2)
print(laburpena)
```

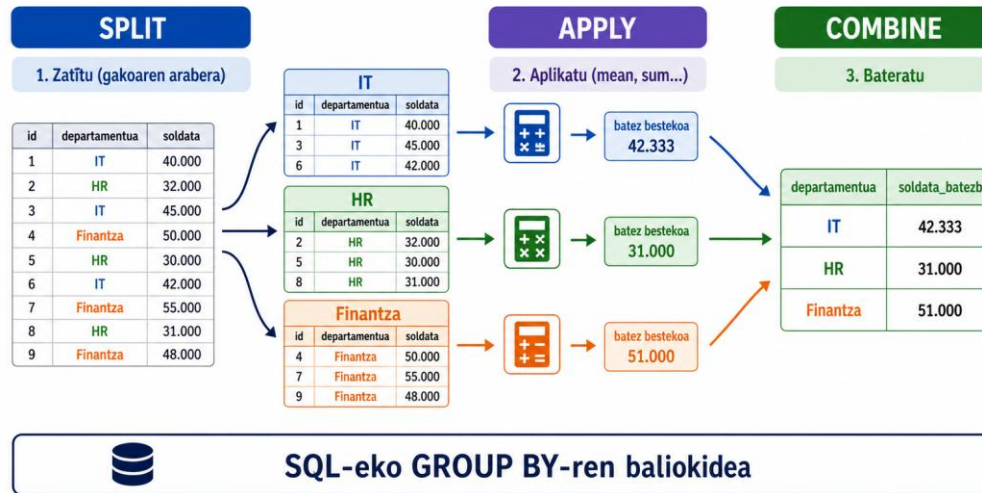
Batzuetan ez dugu taula laburtua nahi, baizik eta jatorrizko taulan zutabe berriak gehitu, talde-informazioa kontuan hartuta. Horretarako **transform** metodoa erabiltzen da, talde-eragiketaren emaitza errenkada bakoitzari hedatzen duena (ezaugarri-ingeniaritzan oso erabilia). Eta **pivot_table** Excel-eko taula dinamikoen baliokidea da: errenkadak gako batek definitzen ditu, zutabeak beste batek, eta gelaxkek hirugarren zutabe baten agregazioa.

```
import pandas as pd

# transform: talde-batezbestekoa errenkada bakoitzari gehitu
df["dep_batezb"] = df.groupby("departamentua")["soldata"].transform("mean")
df["proportzioa"] = df["soldata"] / df["dep_batezb"]

# pivot_table: taula dinamikoa
taula = pd.pivot_table(
    df,
    values="soldata",
    index="departamentua",    # Errenkadak
    columns="hiria",          # Zutabeak
    aggfunc="mean",
    fill_value=0
)
print(taula)
```

groupby: split - apply - combine



9. Irudia: `groupby`-ren split-apply-combine eredua bisualki — taula bat taldeetan banatu (split), agregazioa aplikatu (apply), eta emaitza bateratu (combine).

Q Adibidea: Salmenta-txostena marketin-sailerako Sarrerako eszenara itzuliz: marketin-taldeak hiriko eta kategoriako salmentak nahi ditu.

`df.groupby("hiria")["salmenta"].sum()`-ekin hiriko guztizkoa lortzen da; `pivot_table`-rekin hiri × kategoria gurutze-aula bat eraikitzen da klik bakar batean. Excel-en taula dinamiko bat eskuz eraiki beharrean, kode-lerro bat eta script gisa errepika daiteke datu berriekin. Hilabete bakoitzean txosten bera segundotan.

Idea nagusia `groupby` "split-apply-combine" da: zatitu, aplikatu, bateratu. Taula laburtua nahi baduzu, `groupby().agg()`; jatorrizko taulara emaitza eraman nahi baduzu, `transform`; bi dimentsioko gurutze-aula nahi baduzu, `pivot_table`. SQL-en `GROUP BY` ezagutzen baduzu, hau berehala ulertuko duzu.

Ariketa 2.4 · 20 min · banaka Egoera: Salmenta-DataFrame bat duzu (hiria, kategoria, produktua, salmenta). **Eginkizuna:** 1. Kalkulatu hiri bakoitzeko salmenta osoa (`groupby` + `sum`). 2. Kalkulatu kategoria bakoitzeko batezbestekoa, gehienekoa eta kopurua (`agg`). 3. Sortu `pivot_table` bat: errenkadak hiriak, zutabeak kategoriak, balioak salmenta-batura. 4. Identifikatu zein hiri-kategoria konbinazioak duen salmentarik handiena.

2.5 Taulen Batzea: merge eta concat

Datu-multzo errealak askotan taula desberdinetan zatituta egoten dira (eredu erlazionalaren printzipioa: langileak taula batean, departamentuak beste batean). Ezinbestekoa da Pandas-en taulak elkartzeko bi teknika nagusiak ezagutzea: `concat` (taulak pilatzeko) eta `merge` (SQL-eko JOIN-en baliokidea, gako baten arabera errenkadak parekatzeko).

`concat` egitura bereko taulak pilatzeko erabiltzen da, bertikalki (`axis=0`, errenkadaz errenkada) edo horizontalki (`axis=1`, zutabez zutabe). `merge`, aldiz, SQL-eko `JOIN` eragiketaren baliokidea da: bi taula gako baten arabera elkartzen ditu. Lau elkarketa-mota nagusiak ditu: `inner` (bietan dauden gakoak soilik), `left` (ezkerreko taula osoa), `right` (eskuineko taula osoa) eta `outer` (bi taulen elkartze osoa).

```
import pandas as pd

langileak = pd.DataFrame({
    "id": [1, 2, 3, 4],
    "izena": ["Ane", "Mikel", "Leire", "Jon"],
    "departamentu_id": [10, 20, 10, 30]
})

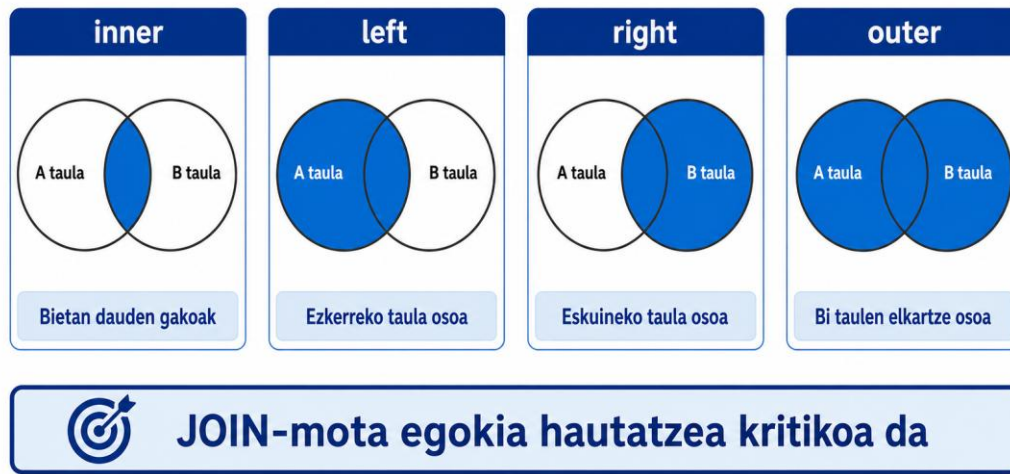
departamentuak = pd.DataFrame({
    "id": [10, 20, 30],
    "izena": ["IT", "HR", "Finantza"],
    "lokazio": ["Bilbo", "Donostia", "Gasteiz"]
})

# INNER JOIN: bi tauletan dauden errenkadak bakarrik
emaitza = pd.merge(
    langileak, departamentuak,
    left_on="departamentu_id", right_on="id",
    how="inner",
    suffixes=("_langilea", "_dep")
)

# concat: taulak pilatu (bertikalki)
df_bateratua = pd.concat([langileak, langileak], ignore_index=True)
```

`left_on` eta `right_on` elkarketa-zutabeen izenak desberdinak direnean erabiltzen dira. Bi tauletan zutabe-izen berdinak agertzen direnean (`izena` adibidean), `suffixes` parametroak atzizkiak gehitzen ditu nahasketa ekiditeko. JOIN-mota egokia hautatzea kritikoa da: `inner`-ek parekatzen ez direnak galtzen ditu; `left`-ek ezkerreko guztiak mantentzen ditu (parekatzen ez direnak NaN-rekin).

merge: JOIN motak



10. Irudia: Lau JOIN-moten konparaketa Venn-diagrama gisa — inner, left, right eta outer, bakoitzak zein errenkada mantentzen dituen erakutsiz.

Q Adibidea: Salmentak eta produktu-katalogoa lotzea Online denda batek bi taula ditu: salmenta-erregistroak (produktu-id bakarrik dute) eta produktu-katalogoa (id, izena, kategoria, prezioa). Txosten batek produktu-izenak behar ditu, ez id-ak. *merge*-rekin bi taulak *produktu_id* gakoaren arabera lotzen dira *left* JOIN batekin (salmenta guztiak mantenduz). Eraitza: salmenta bakoitzak bere produktu-izen eta kategoria. SQL-en *JOIN* bat egiten dakienak berehala ulertzen du.

! Ideia nagusia *concat* = taulak pilatu (forma berekoak); *merge* = taulak gako baten arabera elkartu (SQL JOIN bezala). JOIN-mota da erabaki kritikoa: *inner* parekatzen direnak soilik, *left* ezkerreko guztiak. Zalantzan, marraztu bi taulak paperean eta erabaki zein errenkada behar dituzun eraitzan.

⚠ Kontuz: merge-ak errenkadak bikoizten ditu Gako-zutabeen balio bikoiztuak badaude, *merge*-ak errenkadak biderkatu ditzake ustekabeen (gako bakoitzeko konbinazio guztiak sortuz). Adibidez, produktu-id berarekin bi prezio-erregistro badaude, salmenta bakoitza bi aldiz agertuko da. Beti egiaztatu eraitzaren *shape* merge baten ondoren: errenkada-kopurua espero zenuena al da?

□ **Ariketa 2.5 · 25 min · taldeka Egoera:** Bi DataFrame: bezeroak (id, izena, hiri_id) eta hiriak (id, hiri_izena, probintzia). **Eginkizuna:** 1. Egin *inner* JOIN bat bezeroei haien hiri-izena eransteko. 2. Egin *left* JOIN bat: zer gertatzen da hiri_id-rik gabeko bezeroekin? 3. Sortu bi salmenta-aula (urtarrila, otsaila) eta batu *concat*-ekin. 4. Eztabaidatu: noiz erabili *merge* eta noiz *concat*?

3. Bistaratzeta: Matplotlib eta Seaborn

Datuen bistaratzea analisi esploratzailearen funtsezko zatia da: estatistika hutsek erakuts ez ditzaketen patroiak, joerak eta balio anormalak agerian uzten ditu. Atal honetan Python-en bisualizazio-liburutegi nagusi biak landuko ditugu: **Matplotlib** (oinarrizko liburutegia, kontrol osoa eskaintzen duena) eta **Seaborn** (Matplotlib-en gainean eraikia, estatistika-grafiko ohikoak askoz errazago sortzeko). Bistaratzeta ez da soilik grafikoak sortzea, baizik eta benetan ulergarriak eta esanguratsuak diren irudikapenak diseinatzea.

3.1 Matplotlib: Oinarriak

Matplotlib Python-en grafiko-liburutegi nagusia da. Grafikoen kontrol osoa ematen du, baina kode gehiago eskatzen du Seaborn-ekin alderatuta. Beste bisualizazio-liburutegi gehienek (Seaborn, Pandas-en grafikoak) Matplotlib erabiltzen dute barnean, beraz funtsezkoa da haren oinarriak ulertzea.

Bi kontzeptu nagusi: **Figure** (marrazketa osoa, grafikoak dituen "orria") eta **Axes** (grafiko bakarra, bere ardatzekin eta datuekin). Bereizketa hau modu esplizituan erabiltzeak (`fig, ax = plt.subplots()`) kontrol gehiago eta kode garbiagoa ematen du. Grafiko baten eraikuntza pausoz pauso: figura sortu, datuak gehitu, titulua eta etiketak ezarri, legenda gehitu, eta gorde.

```
import matplotlib.pyplot as plt
import numpy as np

fig, ax = plt.subplots(figsize=(8, 5))

x = np.linspace(0, 2 * np.pi, 100)
ax.plot(x, np.sin(x), label="sin(x)", color="blue", linewidth=2)
ax.plot(x, np.cos(x), label="cos(x)", color="red", linestyle="--")

ax.set_title("Sinu eta kosinua", fontsize=14)
ax.set_xlabel("x (erradianetan)")
ax.set_ylabel("Balioa")
ax.legend()
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("grafikoa.png", dpi=150)
plt.show()
```

Sarritan grafiko bakar baten ordeaz hainbat erlazioeratu erakutsi nahi dira aldi berean, dashboard antzeko ikuspegiak sortzeko. `plt.subplots(errenkadak, zutabeak)`-ekin sareta batean grafiko anitz antolatzen dira. Grafiko-mota egokia datu-motarekin bat etorri behar da: **lerroa** joerretarako, **barrak** konparaketetarako, **sakabanaketa** erlazioetarako, eta **histograma** banaketetarako.

```
import matplotlib.pyplot as plt
import numpy as np

fig, axs = plt.subplots(2, 2, figsize=(12, 8))

x = np.linspace(0, 10, 100)
axs[0, 0].plot(x, np.sin(x), "b-") # Lerro-grafikoa
axs[0, 0].set_title("Lerroa: joerak")

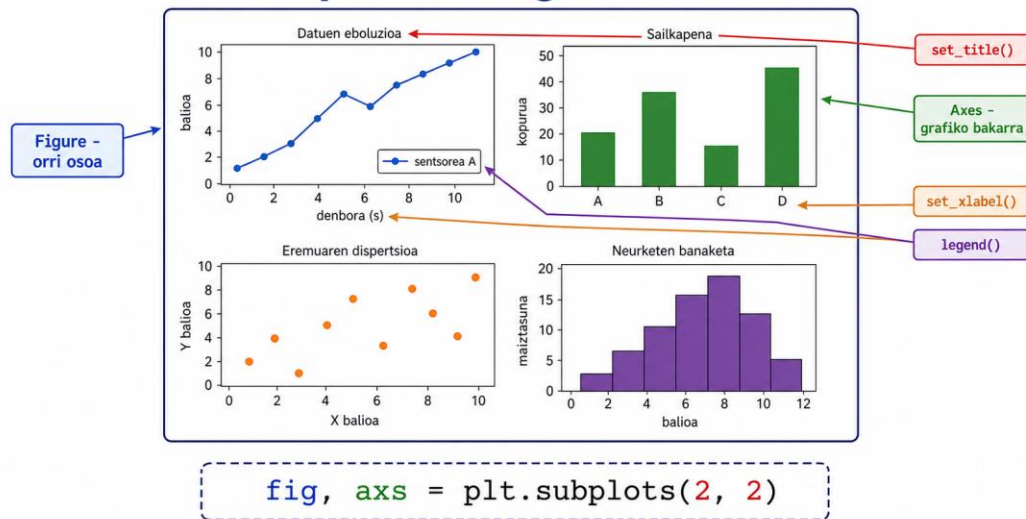
axs[0, 1].bar(["A", "B", "C"], [23, 45, 12]) # Barra-grafikoa
axs[0, 1].set_title("Barrak: konparaketak")

axs[1, 0].scatter(np.random.randn(50), np.random.randn(50), alpha=0.7) # Sakabanaketa
axs[1, 0].set_title("Sakabanaketa: erlazioak")

axs[1, 1].hist(np.random.normal(50, 15, 1000), bins=30) # Histograma
axs[1, 1].set_title("Histograma: banaketa")

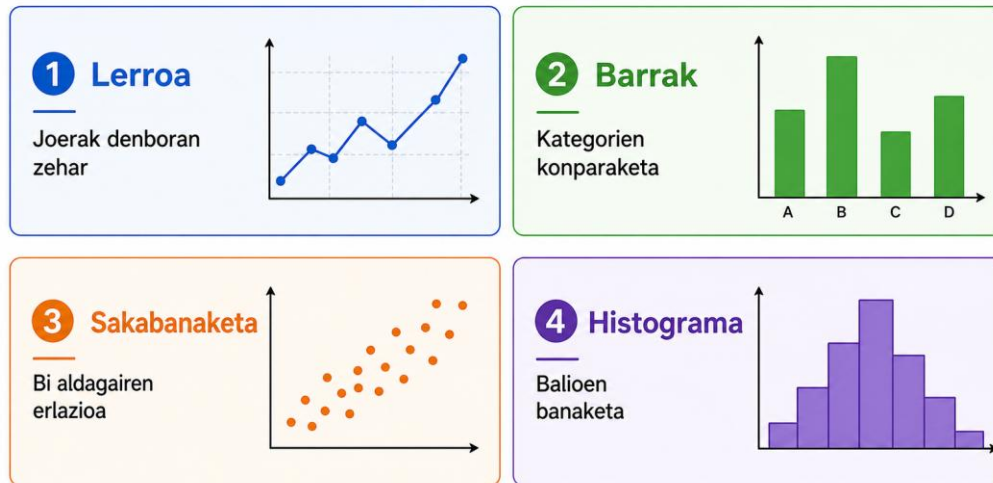
plt.suptitle("Grafiko motak", fontsize=16)
plt.tight_layout()
plt.show()
```

Matplotlib: Figure eta Axes



11. Irudia: Matplotlib-en Figure (orri osoa) eta Axes (grafiko individualak) egitura, 2x2 subplots sareta batekin, elementuak (titulua, ardatzak, legenda) etiketatuta.

Zein grafiko, noiz?



12. Irudia: Lau grafiko-mota nagusiak eta noiz erabili — lerroa (joerak), barrak (konparaketak), sakabanaketa (erlazioak), histograma (banaketa), adibide bisual txikiekin.

Q Adibidea: Hileko salmenten dashboard-a zuzendaritzarako Zuzendaritzak hileko txosten bisuala nahi du: salmenten bilakaera (lerroa), hiriko salmentak (barrak), publizitatea vs salmentak erlazioa (sakabanaketa) eta erosketak-balioen banaketa (histograma). `plt.subplots(2, 2)`-ekin lau grafikoak orri bakar batean antolatzen dira, PDF gisa esportatzeko prest. Bilera bakoitzerako, script bera datu berriekin: dashboard profesionala minututan.

💡 Ideia nagusia Matplotlib-en gakoa Figure/Axes egitura ulertzea da: `fig`, `ax = plt.subplots()`. Eta grafiko-mota datuari egokitzea: lerroa joeretarako, barrak konparaketetarako, sakabanaketa erlazioetarako, histograma banaketetarako. Mota okerrak mezua ezkututzen du.

⚠ Kontuz: tarta-grafikoak gehiegi erabiltzen dira Tarta-grafikoak (*pie*) ezagunak dira, baina askotan irakurtzeko zailak: gizakiok ez ditugu ondo konparatzen angeluak. Bost kategoria baino gehiagorekin, edo balio antzekoekin, barra-grafiko bat ia beti argiagoa da. Erabili tarta-grafikoa kategoria gutxi eta proportzio oso desberdinekin soilik.

□ Ariketa 3.1 • 25 min • banaka Egoera: Klase bateko datuak dituzu: ikasleak, notak, klaseko ordu-kopurua. **Eginkizuna:** 1. Sortu barra-grafiko bat ikasleen notekin. 2. Sortu sakabanaketa-grafiko bat: ordu-kopurua (*x*) vs nota (*y*). Ikusten al duzu erlazioaren bat? 3. Sortu 2×2 subplots bat lau grafiko motarekin. 4. Gorde emaitza PNG eta PDF formatuetan (`savefig`).

3.2 Seaborn: Grafiko Estatistikoak

Seaborn Matplotlib-en gainean eraikita dago eta grafiko estatistikoak askoz errazago egiteko diseinatu da. Pandas DataFrame-ekin zuzenean lan egiten du, eta defektuzko gai estetiko atseginak ditu, beraz emaitza profesionalagoa lortzen da kode gutxiagorekin. Seaborn-ek ez du Matplotlib ordezkatzen, baizik eta osatzen du.

Seaborn-en ikur bereizgarria **hue parametroa** da: aldagai kategoriko baten arabera datuak koloreka sailkatzen ditu, beste dimentsio bat eransten dio bisualizazioari. Hauxe Pandas-en **groupby** egitearen baliokide bisuala da. Hiru grafiko-talde nagusi: **banaketa-grafikoak** (**histplot**, **boxplot**, **violinplot**), **erlazio-grafikoak** (**scatterplot**, **regplot**, **pairplot**) eta **kategorien arabekoak** (**barplot**, **countplot**).

```
import seaborn as sns
import matplotlib.pyplot as plt

# Seaborn-ek datu-multzo erabilgarriak ditu
df = sns.load_dataset("tips") # Jatetxe-propinen datu-multzoa

# Histograma + KDE (banaketa estimazioa), sexuaren arabera
sns.histplot(data=df, x="total_bill", hue="sex", bins=20, kde=True)
plt.show()

# Boxplot: banaketaren kuartilak eta atipikoak
sns.boxplot(data=df, x="day", y="total_bill", hue="sex")
plt.show()

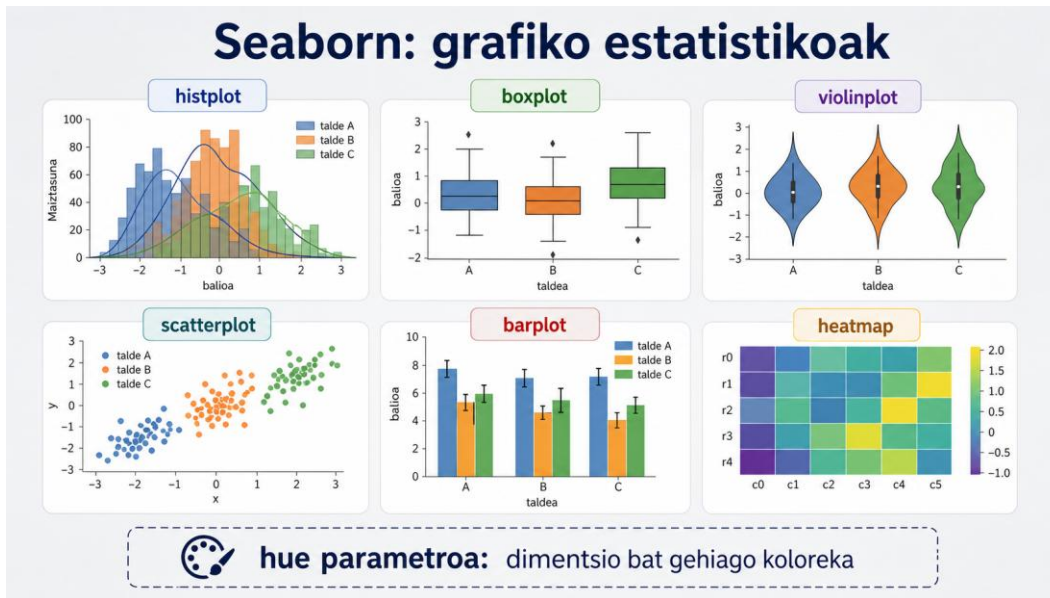
# Sakabanaketa: bi aldagairen arteko erlazioa, dimentsio gehiagorekin
sns.scatterplot(data=df, x="total_bill", y="tip", hue="time", size="size")
plt.show()
```

Datu-zientzialari batek etengabe erabiltzen duen tresna **bero-mapa (heatmap)** da, batez ere **korrelazio-matrizeak** erakusteko. Korrelazioak bi aldagairen arteko erlazioaren indarra neurtzen du (-1etik 1era): zenbat eta gertuago 1etik (edo -1etik), orduan eta lotura estuagoa. Hau ML-ren ezaugarri-aukeraketan funtsezkoa da.

```
import seaborn as sns
import matplotlib.pyplot as plt

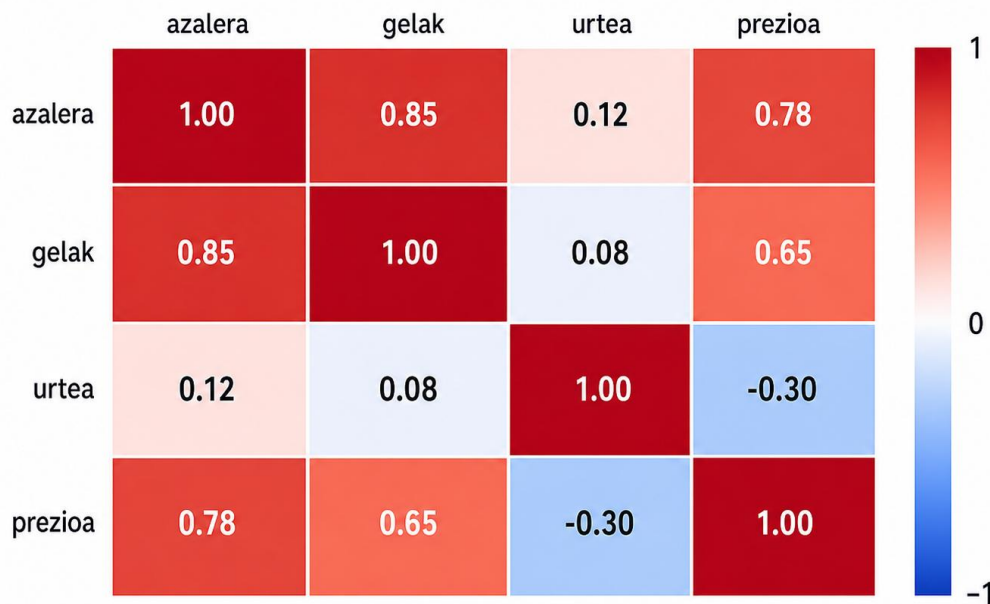
df = sns.load_dataset("tips")

# Korrelazio-matrizea heatmap gisa
korr = df[["total_bill", "tip", "size"]].corr()
sns.heatmap(korr, annot=True, fmt=".2f", cmap="coolwarm",
            vmin=-1, vmax=1, square=True)
plt.title("Korrelazio-matrizea")
plt.show()
```



13. Irudia: Seaborn-en grafiko-mota nagusien galeria — histplot, boxplot, violinplot, scatterplot, barplot eta heatmap, hue parametroaren erabilerarekin.

Korrelazio-matrizea



Gorria = korrelazio positiboa, **urdina** = negatiboa

14. Irudia: Korrelazio-matrize baten beroketa-mapa, coolwarm paletarekin (gorria positibo, urdina negatibo), balioak gelaxketan idatzita -1etik 1era.

Q Adibidea: Etxebizitza-prezioen analisisa inmobiliaria batean Inmobiliaria batek etxebizitzaren datuak ditu: azalera, gela-kopurua, eraikuntza-urtea, prezioa. Zein faktorek eragiten du gehien prezioan? `df.corr()` + `heatmap`-ekin korrelazio-matrize bat eraikitzen da: azalera-prezio korrelazioa 0.85 da (oso indartsua), eraikuntza-urte-prezio korrelazioa 0.12 (ahula). Begi-kolpe batean ikusten da azalera dela faktore nagusia. ML eredu bat eraiki aurretik, `heatmap`-ak ezaugarri garrantzitsuenak erakusten ditu.

! Ideia nagusia Seaborn = Matplotlib errazagoa eta estatistikoa. `hue` parametroa da gakoa: dimentsio bat gehiago koloreka. Korrelazio-heatmap-a tresna ezin hobea da ML-ren aurretik: zein aldagaik daukan loturarik handiena helburuarekin begi-kolpe batean erakusten du. Korrelazioa ez da kausalitatea, baina hari ona da.

☐ **Hausnartzeko** Korrelazioak ez du esan nahi kausalitatea. Adibidez, izozki-salmentak eta itotze kasuak korrelazionatuta daude (biak udan gertatzen dira), baina izozkiak ez dira itozeen kausa. Pentsa: zein adibide ezagutzen dituzu non bi gauza korrelazionatuta dauden baina bata ez den bestearen kausa?

☐ **Ariketa 3.2 · 25 min · banaka Egoera:** `sns.load_dataset("tips")` datu-multzoa erabiliz. **Eginkizuna:** 1. Sortu propinen (`tip`) histograma bat, egunaren arabera koloreztatuta (`hue`). 2. Sortu boxplot bat: faktura (`total_bill`) egunaren arabera. 3. Sortu sakabanaketa bat: faktura vs propina, ordu-tartearen arabera koloreztatuta. 4. Sortu korrelazio-heatmap bat zenbakizko zutabeekin. Zein bi aldagai daude korrelazionatuak?

3.3 Konfigurazio eta Esportazioa

Behin grafikoak sortzeko gai garenean, hurrengo urratsa haien itxura pertsonalizatzea eta hainbat formatutan esportatzea da. Itxura koherentea (eta enpresa-marka edo argitalpenaren estilora egokitua) garrantzitsua da txosten profesionaletan.

Matplotlib eta Seaborn-ek estilo-aukera anitz dakartzate. Estiloa proiektu-mailan zehaztea ohitura ona da. Garrantzitsua: estilo eta kolore-paleta aukeratzean, **irisgarritasuna** kontuan hartu: kolore-itsutasuna duten erabiltzaileek paleta sinpleak hobeto ulertzen dituzte.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Estiloak eta gaiak
plt.style.use("seaborn-v0_8")
sns.set_theme(style="whitegrid")      # Sare zuriko hondoa
sns.set_palette("muted")             # Kolore lasaiak
sns.set_theme(font_scale=1.4)        # Letra handiagoa aurkezpenetarako
```

Grafikoak fitxategi-formatu desberdinetan gordetzen dira erabileraren arabera: **PNG/JPEG** web edo aurkezpenetarako, **PDF/SVG** inprimaketa edo bektore-edizioarako. `savefig`-en parametro

garrantzitsuak: `dpi` (bereizmena), `bbox_inches="tight"` (zuriunearen mozketaren egokia), `transparent` (hondo gardena).

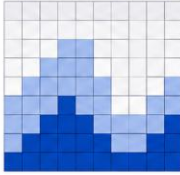
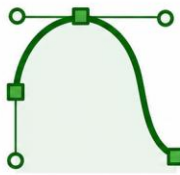
```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(10, 6))
ax.plot([1, 2, 3], [4, 5, 6])

fig.savefig("grafikoa.png", dpi=150, bbox_inches="tight") # Web/aurkezpena
fig.savefig("grafikoa.pdf", bbox_inches="tight")          # Txostenak
fig.savefig("grafikoa.svg", bbox_inches="tight")          # Eskala aldagaria
```

Bektore-formatuek (SVG, PDF) edozein tamainatara handitzeko aukera ematen dute kalitatea galdu gabe; raster-formatuek (PNG, JPEG) tamaina finkoa dute baina arinagoak dira web-erako. Web-aplikazioetan (Flask, FastAPI, Streamlit), grafikoak memorian gorde daitezke (`io.BytesIO`) diskoan idatzi gabe.

Grafikoak gordetzeko formatuak

Raster (PNG, JPEG)	Bektore (PDF, SVG)
 - Tamaina finkoa - Arina, web-erako - Handitzean pixelatua	 - Eskala aldagaria - Kalitatea ez galdu - Editagarria
 Use case: Web, aurkezpenak	 Use case: Inprimaketa, txostenak

 `savefig(..., bbox_inches='tight')` ia beti

15. Irudia: Grafiko-esportazioko formatuen konparaketa — PNG/JPEG (raster, web), PDF/SVG (bektore, inprimaketa), bakoitzaren erabilera-kasuak eta ezaugarriak.

Q Adibidea: Txosten korporatiboa bezeroarentzat Aholkularitza-enpresa batek txosten bat prestatu behar du bezero batentzat, marka korporatiboarekin. Grafikoak estilo koherentean sortzen dira (`sns.set_theme` + kolore-paleta korporatiboa), eta PDF gisa esportatzen dira `bbox_inches="tight"`-ekin, txostenean txertatzeko. Aurkezpenerako, PNG bertsioa `dpi=150`-ekin. Estilo bera proiektu osoan: irudi profesional eta koherentea.

! Ideia nagusia Estiloa proiektu-mailan zehaztu (`sns.set_theme`) koherentziarako. Esportazioan: PNG web/aurkezpenetarako, PDF/SVG inprimaketa edo edizio-kalitaterako. `bbox_inches="tight"` ia beti, zuriune alferrak kentzeko. Irisgarritasuna ez ahaztu: paleta sinpleak eta letra-tamaina egokia.

□ **Ariketa 3.3 · 15 min · banaka Egoera:** Aurreko ariketako grafikoak erabiliz.
Eginkizuna: 1. Aldatu Seaborn gaia (*whitegrid*, *darkgrid*) eta kolore-paleta (*muted*, *husl*). Zein gustatzen zaizu gehien? 2. Igo letra-tamaina aurkezpenetarako (*font_scale=1.4*). 3. Gorde grafiko bat hiru formatutan (PNG, PDF, SVG) eta konparatu fitxategi-tamainak. 4. Eztabaidatu: zein formatu erabiliko zenuke web-orri baterako? Eta inprimatutako txosten baterako?

4. Datuen Bertsio Kontrola: DVC

Atal honetan datuen bertsio-kontrola lantzen duen **DVC** tresna aurkeztuko dugu modu laburrean. 1. gaian Git ikasi genuen kodea bertsionatzeko; orain ikusiko dugu ML-proiektuetan kodea bertsionatzea ez dela nahikoa: datuek ere bertsionatu behar dute esperimenduak erreproduzigarriak izateko. DVC-ren tratamendu sakon bat MLOps eremuko da (3. gaian landuko da); hemen oinarriak ezagutuko ditugu.

4.1 DVC zer da eta zergatik beharrezkoa den

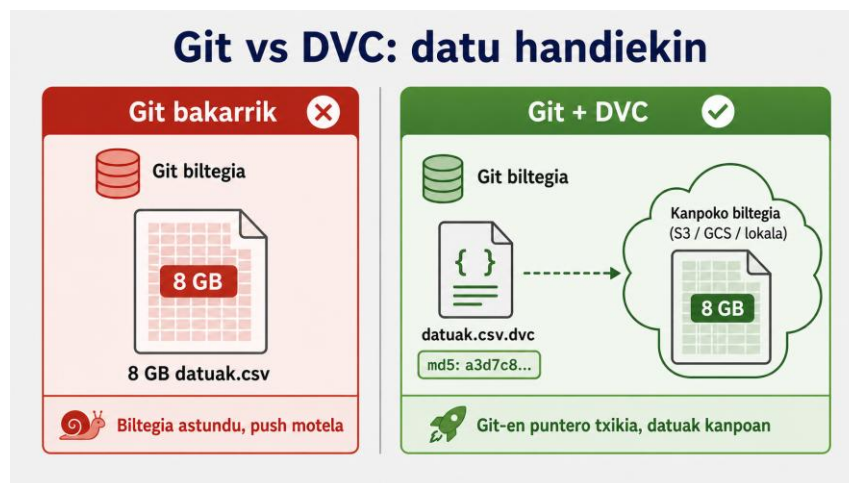
DVC (Data Version Control) ML-proiektuetako datuak eta ereduak bertsionatzeko sistema da. Git-ekin duen antzekotasuna ikusita, sarritan "**datu-zientzialarientzako Git**" deitzen zaio. Baina zergatik ez da Git nahikoa?

Git **kodereantzat** diseinatua dago: testu-fitxategi txikiak, lerroz lerro aldatzen direnak. Datu-fitxategi handiak (GB-ak edo TB-ak) ezin dira Git-en bidez modu eraginkorrean kudeatu: biltegia astundu, push-ek minutuak hartu, clone-ek dena birdeskargatu. DVC-k honela konpontzen du: fitxategi handiak **kanpoko biltegian** gordetzen ditu (S3, Google Cloud, biltegi lokala...), eta Git-en **puntero txiki bat** (`.dvc` fitxategia, hash bat duena) gordetzen da. Horrela, Git arina mantentzen da eta datuak eskalagarriki kudeatzen dira.

Ohiko fluxua DVC-rekin:

```
dvc add datuak.csv      # datuak.csv.dvc sortzen da (puntero txikia)
git add datuak.csv.dvc  # Puntero txikia Git-era (ez datu-fitxategia)
dvc push                # Datu handiak kanpoko biltegiara
```

Gakoa argia da: Git-en hash-puntero txikia, kanpoko biltegian datu-fitxategi handia. Datu-bertsioak kode-bertsioak bezala jarrai daitezke, eta esperimendu bakoitza zein datu-bertsioekin egin zen dokumentatuta geratzen da.



16. Irudia: Git-en mugak datu handiekin vs DVC-ren konponbidea — Git-ek puntero txikia gordetzen du, datu handia kanpoko biltegian (S3/GCS/lokala) geratzen da.

Q Adibidea: Eredu-entrenamendua erreproduzitzea Datu-zientzialari batek eredu bat entrenatu zuen duela hiru hilabete, %94 zehaztasunarekin. Orain bezeroak eredu hori berregitea eskatzen du, baina datuak aldatu egin dira bitartean. Git-ek kodea gordetzen du, baina ez datuen orduko bertsioa. DVC-rekin, `git checkout` + `dvc checkout`-ekin orduko kode eta datu-bertsioa berreskuratzen dira, eta eredia zehazki berregiten da. Erreproduzigarritasuna = konfiantza zientifikoa eta enpresariala.

! Ideia nagusia Git kodearentzat, DVC datuentzat. DVC-k datu handien puntero txikia bakarrik gordetzen du Git-en, datu-fitxategi erraldoia kanpoko biltegian utzita. Horrela ML-proiektuak erabat erreproduzigarriak dira: kodea + datuak + emaitzak, dena bertsionatuta.

⚠ Kontuz: DVC ez da Git-en ordezkoa DVC-k Git-EKIN batera lan egiten du, ez haren orde. Kodea Git-en bertsionatzen jarraitzen duzu; DVC datu eta eredu handietarako da soilik. Biak elkarrekin: `git push` kodearentzat, `dvc push` datuentzat. Ez nahastu: `.dvc` erakusle-fitxategiak Git-en doaz, datu errealak ez.

□ Hausnartzeko Imajinatu enpresa batean ML eredu bat ekoizpenean dagoela, eta bezero batek erabakiaren inguruan kexa jartzen duela ("zergatik ukatu zenidaten mailegua?"). Erabakia auditatzeko, zein datu eta zein eredu-bertsioarekin hartu zen jakin behar da. Zer gertatuko litzateke datuak bertsionatuta ez baleude?

4.2 DVC oinarritzko erabilera

DVC `pip` bidez instalatzen da. Kanpoko biltegi-mota desberdinetarako (S3, Google Cloud Storage, Azure) gehigarri espezifikoak instalatu behar dira korxeten artean.

```
pip install dvc
pip install "dvc[s3]"          # Amazon S3
pip install "dvc[gs]"         # Google Cloud Storage
```

DVC Git biltegi baten gainean lan egiteko diseinatuta dago. Beraz, lehenik Git hasieratu (`git init`), eta gero `dvc init`. Ondoren, datu-fitxategiak DVC kontrolpean jartzen dira `dvc add`-ekin: honek fitxategiaren hash-a kalkulatu du, erakusle-fitxategia sortzen du eta jatorrizkoa `.gitignore`-ra gehitzen du automatikoki.

```
# Proiektua hasieratu
git init
dvc init
git commit -m "chore: DVC hasieratu"

# Datu-fitxategi bat DVC kontrolpean jarri
dvc add data/train.csv          # train.csv.dvc erakusle-fitxategia sortzen da

# Puntero-fitxategia Git-era (datu-fitxategia EZ)
git add data/train.csv.dvc data/.gitignore
```

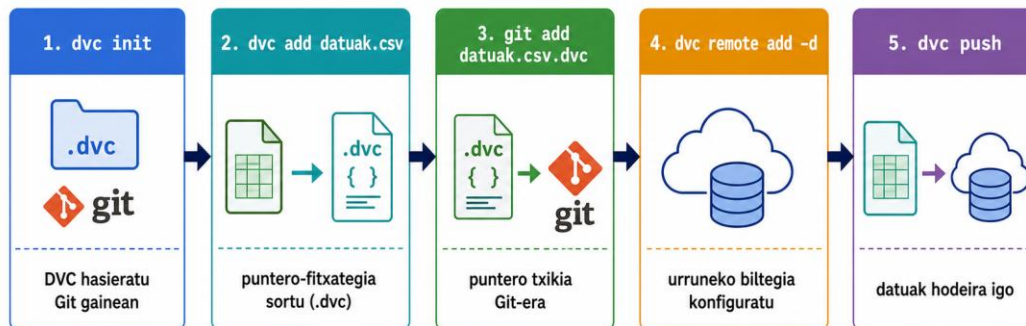
```
git commit -m "data: entrenamenduko datu-multzoa gehitu"
```

Datuak hodeira igotzeko, urruneko biltegi bat konfiguratzeko da (-d aukerak defektukoa dela adierazten du). Ondoren, `dvc push/dvc pull` Git-eko `git push/git pull`-ren baliokideak dira, baina datuentzat.

```
# Urruneko biltegia konfiguratu
dvc remote add -d biltegia s3://nire-bucket/dvc

# Datuak igo/jaitsi
dvc push      # Datuak biltegiara (git push bezala, baina datuentzat)
dvc pull      # Datuak biltegitik (git pull bezala)
```

DVC: oinarritzko lan-fluxua



Datu-fitxategi handia EZ doa Git-era, biltegiara baizik

17. Irudia: DVC-ren lan-fluxua urratsez urrats — `dvc init`, `dvc add` (erakusle-fitxategia), `git add` (puntero Git-era), `dvc remote add` eta `dvc push` (datuak hodeira).

Q Adibidea: Talde banatua datu-multzo handi batekin Hiru datu-zientzialari (Bilbo, Donostia, Gasteiz) eredu berean lan egiten dute, datu-multzoa 8 GB-koa da. Git-ekin bakarrik: ezinezkoa, datuak ezin dira Git-en kabitu. DVC-rekin: datuak S3 biltegi partekatuan daude; bakoitzak `git pull` (kodea + punteroak) eta `dvc pull` (datuak) egiten ditu eta denek bertsio bera dute. Lankide batek datuak eguneratzen dituenean, `dvc push` egiten du eta besteek `dvc pull`-ekin jasotzen dute. Git-en eta DVC-ren konbinazioa = talde-lana datu handiekin.

Idea nagusia DVC-ren fluxua Git-en oso antzekoa da: `dvc add` (Git-en `git add` bezala datuentzat), `dvc push/dvc pull` (urruneko biltegiara/tik). Funtsezkoa: `.dvc` erakusle-fitxategiak Git-en doaz, datu-fitxategi handiak DVC biltegian. Git ezagutzen baduzu, DVC egun batean menderatuko duzu.

□ **Ariketa 4.1 · 20 min · banaka Egoera:** Proiektu txiki bat duzu, datu-fitxategi batekin (*data/notak.csv*). **Eginkizuna:** 1. Hasieratu Git eta DVC (*git init, dvc init*). 2. Jarri *notak.csv* DVC kontrolpean (*dvc add*). 3. Ireki *notak.csv.dvc* fitxategia: zer dago barruan? Zer da *md5* hash-a? 4. Konfiguratu biltegi lokal bat (*dvc remote add -d lokala /tmp/dvc-biltegia*) eta egin *dvc push*. 5. Eztabaidatu: zergatik dago *notak.csv.gitignore*-an baina ez *notak.csv.dvc*?

5. Plataforma Komertzialak: Azure ML, IBM Watson, Knime

Atal honetan, orain arte landu dugun Python ekosistema irekia (NumPy, Pandas, Matplotlib, scikit-learn) **merkatuko plataforma komertzialekin** erlazionatuko dugu. BOEko Errege Dekretuak espresuki aipatzen ditu plataforma hauek IE2 ikaskuntza-emaitzan: Azure ML Studio, IBM Watson, Knime, SPSS Modeler. Ezagutu zein diren; zein erabilera duten enpresetan; eta, nola alderatzen diren iturburu-irekidun alternatibekin, lan-merkatura ateratzean edozein ingurunetan moldatzeko gai izateko.

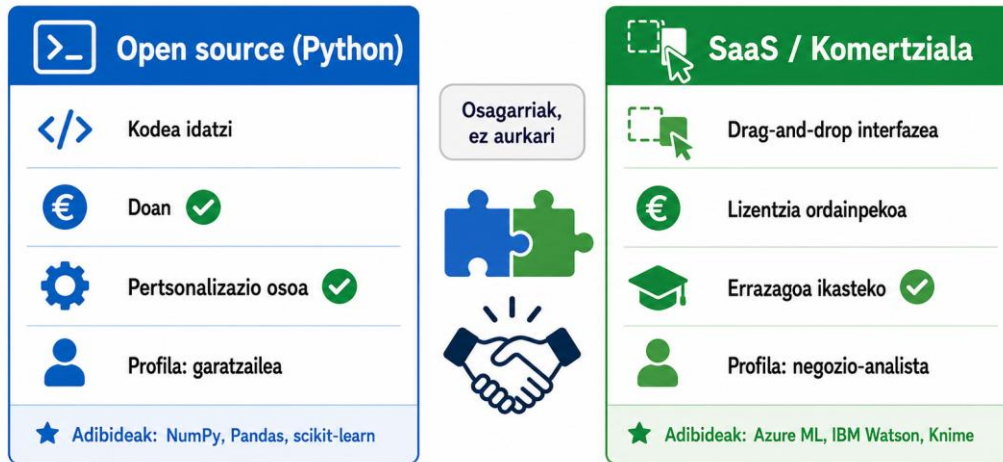
5.1 Open source vs SaaS: bi mundu osagarriak

Datu-zientziaren tresnetan bi familia nagusi daude: **open source** (Python eta haren liburutegiak, kodea idatzi behar dena) eta **SaaS / plataforma komertzialak** (interfaze grafikoa, drag-and-drop, lizentzia ordainpekoak). Funtsezkoa da ulertzea **biak osagarriak direla, ez aurkari**: enpresa askok biak erabiltzen dituzte.

Eredua	Open source (Python)	SaaS / Plataforma komertziala
Adibideak	NumPy, Pandas, scikit-learn	Azure ML, IBM Watson, Knime, SPSS
Erabilera	Garatzaileek kodea idatzi	Drag-and-drop interfaze grafikoa
Hasierako kostua	✓ Doan	✗ Lizentzia ordainpekoak
Pertsonalizazioa	✓ Erabatekoa	□ Plataformaren mugen barruan
Ikasketa-kurba	✗ Aldapatsua (kodea)	✓ Errazagoa (bisuala)
Erabiltzaile-profila	Datu-zientzialaria, ingeniaria	Negozio-analista, citizen data scientist
Lan-merkatua	Tech-enpresak, scale-up-ak	Banketxe, aseguruak, administrazioa

Datu-zientzialariak Python-en lan egiten du gehienetan, malgutasun eta kontrol osoa nahi baititu. Negozio-analista batek, aldiz, Knime edo Power BI nahiago izan dezake, ez baitu programatzen jakin behar. Tekniko bat **bietan moldatzea** balio handikoa da lan-merkaturan: enpresa batzuek Python erabiltzen dute, beste batzuek plataforma komertzialak, eta askok biak.

Open source vs Plataforma komertziala



18. Irudia: Open source (Python, kodea) eta SaaS plataformen (drag-and-drop) bi munduen konparaketa, osagarriak direla erakutsiz, erabiltzaile-profilekin.

! Ideia nagusia Open source eta plataforma komertzialak ez dira aurkari: osagarriak dira. Python kontrolerako eta malgutasunerako; plataforma komertzialak abiadurarako eta profil ez-teknikoentzat. Tekniko on batek bietan moldatzen jakin behar du, lan-merkatuan biak topatuko baititu.

5.2 Azure Machine Learning Studio

Azure ML Studio Microsoft-en plataforma kudeatua da ML eredu osoaren bizi-ziklorako: datuak prestatu, ereduaren entrenamendu, ebaluatu, zabaldu eta monitorizatu. Hodeian dago osorik (PaaS), eta hiru ikuspegi nagusi eskaintzen ditu: kodea (Python/R), no-code (drag-and-drop *Designer*) eta **AutoML**.

Ezaugarri nabarmenenak: **Designer** (lan-fluxu grafikoa, moduluak drag-and-drop bidez konektatuz), **AutoML** (datuak igo, helburua adierazi, eta Azure-k automatikoki probatzen ditu algoritmoak eta hiperparametro onenak), **endpoint kudeatua** (eredua REST API gisa zabaldu klik gutxitan) eta **Azure-ko integrazioa** (Blob Storage, Power BI).

Q Adibidea: Bezero-galera (churn) aurrestea Azure ML-rekin Telefonía-enpresa batek bezero-galera aurrean nahi du. Datu-analista batek CSV bat igotzen du Azure ML-ra → *Designer*-ekin lan-fluxua sortzen du (datuak kargatu → null balioak bete → train/test split → logistic regression → ebaluatu) → *AutoML*-ekin alderatzen du eta gradient boosting ereduak emaitza hobea ematen duela ikusten du → endpoint gisa zabaltzen du → enpresa-aplikazioak hileroko deitzen dio. **Python-kode-lerrorik gabe** egin daiteke prozesu osoa.

! Ideia nagusia Azure ML hiru ikuspegitan moldatzen da: kodea (Python), no-code (*Designer* drag-and-drop) eta *AutoML* (automatikoa). Microsoft ekosistema duten

enpresetarako aukera naturala da, eta kodea eta interfaze grafikoa konbinatzeko aukera ematen du profil desberdinen arabera.

5.3 IBM Watson Studio

IBM Watson Studio datuak prestatu, ereduak eraiki eta zabaltzeko plataforma osoa da. IBM Cloud-en parte da, eta **enpresa-erabiltzaileengan** zentratuta dago bereziki — banku-aseguru-osasun sektoreetan oso erabilia, governance-aren eta GDPR betetzearen garrantziagatik.

Ezaugarri nabarmenenak: **Watson Knowledge Catalog** (enpresa-mailako datu-katalogoa eta governance-a, GDPR betetzeko), **AutoAI** (AutoML-aren IBM-en bertsioa), **SPSS Modeler integratua** (lan-fluxu bisualak hodeian) eta **Watson OpenScale** (eredu-alborapena eta gardentasuna neurtzeko).

Q Adibidea: Mailegu-onarpena banketxe batean Banketxe batek mailegu-onarpenetan IA aplikatu nahi du, baina arautegi zorrotzarekin. Watson Studio-n: datuak Watson Knowledge Catalog-etik kargatu (governance bermatuta) → AutoAI-rekin 50 eredu probatu eta hoberena hautatu (XGBoost) → Watson OpenScale-rekin **alborapena egiaztatu** (sexuagatik edo adinagatik diskriminaziorik ez dagoela bermatu) → REST API gisa zabaldu. Sektore arautuetan, governance eta gardentasuna funtsezkoak dira, eta hor du Watson-ek bere indarra.

💡 Ideia nagusia IBM Watson Studio enpresa handi eta sektore arautuetarako da bereziki (banku, aseguru, osasun). Bere indarra governance-an, GDPR betetzean eta eredu-gardentasunean dago (OpenScale, alborapena neurtzeko). Sektore horietan, "zergatik" erabaki bat hartu den justifikatzea bezain garrantzitsua da zehaztasuna bera.

5.4 Knime Analytics Platform

Knime (Konstanz Information Miner, Alemaniako Konstanz unibertsitatean sortua) datu-zientzia eta ML-rentzako plataforma da, **drag-and-drop interfaze grafikoarekin**. Bertsio doakoa (Knime Analytics Platform) eta enpresa-bertsioa (Knime Server) ditu.

Ezaugarri nabarmenenak: **4000+ nodo** desberdin (CSV irakurri, datuak iragazi, ereduak aplikatu, grafikoa sortu), **Python eta R integrazioa** (lan-fluxuaren erdian skript bat sartu), **doakoa** (oinarrizko plataforma mugarik gabe) eta **ML-kudeaketa** (scikit-learn, H2O, XGBoost nodo bidez). Bioteknologia, farmazia eta ikerketa akademikoan asko erabiltzen da.

Q Adibidea: Bezero-segmentazioa Python jakin gabe Marketin-talde batek bezero-segmentazioa egin nahi du, baina inork ez daki Python-ik. Knime-n: CSV-tik bezeroak kargatu → null balioak bete (*Missing Value* nodoa) → estandarizatu (*Normalizer*) → K-Means klustering aplikatu → emaitzak bistaratu (*Scatter Plot*) → CSV gisa esportatu. Lan-fluxu osoa **interfaze grafikoan eraikitzen da, programaziorik gabe**, nodoak elkarrekin lotuz. Profil ez-teknikoentzat datu-zientzia eskuragarri.

🔗 **Idea nagusia** Knime drag-and-drop bidezko datu-zientzia da: nodoak elkarri lotuz lan-fluxu osoak eraikitzen dira, programaziorik gabe. Doakoa da (oinarrizkoa) eta Python/R integratzen ditu. "Citizen data scientist"-entzat (programatzen ez dakiten profilentzat) aukera bikaina, eta datu-zientzialariekin ere ondo konbinatzen da.

5.5 SPSS Modeler eta beste plataforma klasikoak

SPSS Modeler (jatorriz ClemSPSS, IBM-ek 2009an eresia) datu-zientziaren plataforma tradizionala bat da, 1990eko hamarkadatik. Drag-and-drop lan-fluxuak (*streams*) eraikitzen ditu, eta hainbat sektoretan oso ezaguna da (banku, telekomunikazio, gobernua). Plataforma klasiko hauek lan-merkatuan oraindik oso presente daude, batez ere enpresa handietan.

Plataforma	Hornitzailea	Indargunea
SPSS Statistics	IBM	Estatistika klasikoa, akademian eta osasun-ikerketan
SAS Enterprise Miner	SAS Institute	Banketxe handietan tradizionala
RapidMiner	Altair	Knime-ren lehiakidea, drag-and-drop
DataRobot	DataRobot	AutoML enpresarial heldua
Dataiku DSS	Dataiku	Datu-zientzia kolaboratiboa enpresetarako
Alteryx	Alteryx	Datuen aurreprozesamendua + analitika

Plataforma horiek lan-merkatuko ohiko izenak dira FP-tekniko batentzat enpresa handietan: banku-aseguru-osasun sektoreetan ez da arraroa SPSS, SAS edo DataRobot topatzea Python-en ordeztu (edo paraleloan). Ez ezagutzea ez da arazoa, baina existitzen direla jakitea eta zertarako balio duten ulertzea bai garrantzitsua da.



19. Irudia: Datu-zientziaren plataforma komertzialen panorama — Azure ML, IBM Watson, Knime, SPSS, SAS, DataRobot, bakoitza bere logoarekin eta erabilera-sektorearekin.

💡 **Idea nagusia** SPSS, SAS eta antzeko plataforma klasikoak oraindik oso presente daude enpresa handietan eta sektore tradizionaletan. FP-tekniko batek ez ditu menderatu behar, baina ezagutu bai: existitzen direla, drag-and-drop direla, eta non topa daitezkeen (banku, osasun, gobernu). Lan-merkatuan moldatzeko ezagutza-mapa bat da.

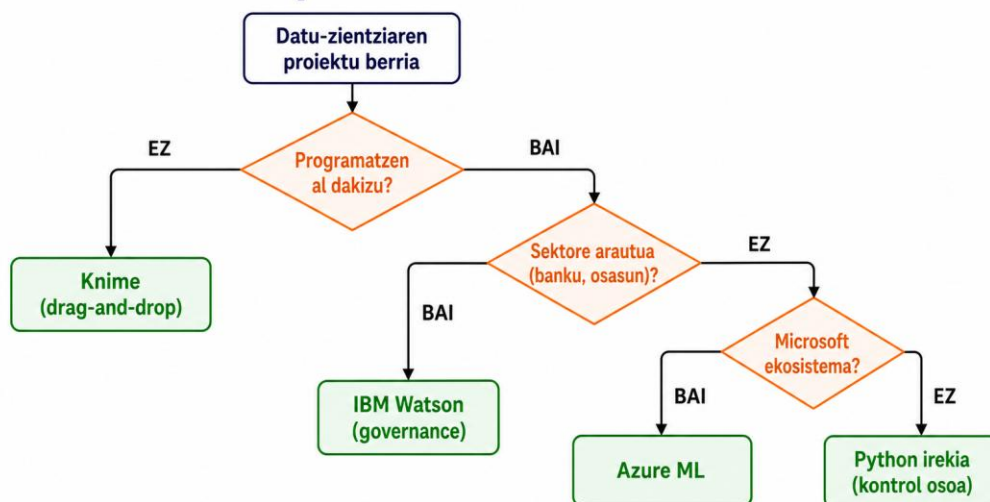
5.6 Konparaketa eta zer aukeratu

Plataforma guztiak alderatzeko, hona hemen taula erabakigarri bat. Gakoa ez da "zein den onena" abstraktuan, baizik eta "zein den egokiena testuinguru bakoitzerako": enpresaren ekosistema, taldearen profila eta proiektuaren beharrak.

Irizpidea	Python irekia	Azure ML	IBM Watson	Knime	SPSS Modeler
Profila	Garatzaile, ingeniaria	Hibridoa	Enpresa-analista	Citizen DS	Estatistika klasikoa
Hasierako kostua	✓ Doan	☐ Hodei-erabilera	✗ Hodei + lizentzia	✓ Doan (oinarriz.)	✗ Lizentzia handia
Pertsonalizazioa	✓ Erabatekoa	✓ Altua	☐ Ertaina	☐ Ertaina	✗ Baxua
Programatu beharra	✓ Bai	☐ Aukeran	☐ Aukeran	✗ Ez	✗ Ez
Governance/GDPR	☐ Eskuz	✓ Natiboa	✓ Bikaina	☐ Ertaina	✓ Ona
Noiz aukeratu	Talentu teknikoa, kontrol osoa	Microsoft ekosistema	Sektore arautuak	Profil ez-teknikoak	Sektore klasikoak

Praktikan: enpresa modernoek askotan **Python + hodeiko plataforma kudeatu bat** konbinatzen dute. Tradizionalek SPSS edo SAS daukate oraindik. FP-tekniko batek lan-merkatura ateratzean Python-en lana topatuko du gehienetan, baina ez da arraroa Azure ML edo Watson erabiltzea ere. **Bietan moldatzea oso baliotsua da.**

Zein plataforma aukeratu?



20. Irudia: Plataforma egokia aukeratzeko erabaki-zuhaitza — galderen arabera (programatzen al dakizu? zein ekosistema? sektore arautua?) plataforma egokira gidatzen duena.

Q Adibidea: Hiru enpresa, hiru aukera - Tech startup bat Bilbon → Python + Azure ML. Garatzaileek kodea idazten dute, baina ekoizpenerako Azure-ren MLOps-a erabiltzen dute. - **Aseguru-etxe handi bat Madrilen** → IBM Watson. Governance eta GDPR kritikoak dira; OpenScale-ekin erabakiak auditatzen dituzte. - **Bioteknologia-enpresa bat Donostian** → Knime + Python. Biologoek Knime erabiltzen dute drag-and-drop, datu-zientzialariek Python skriptak txertatzen dituzte lan-fluxuan.

! Ideia nagusia Ez dago plataforma "onenik" abstraktuan; egokiena testuingurura egokitzen dena da. Galdetu: programatzen al dakigu? zein ekosistema (Microsoft, IBM...)? sektore arautua al da? profil teknikoa ala negoziokoa? Erantzunek gidatzen dute aukeraketa. Eta askotan, erantzuna "biak" da.

□ **Ariketa 5.1 · 30 min · taldeka Egoera:** Hiru enpresa fikzio aurkezten dizkizute, bakoitzak datu-zientziaren proiektu bat hasi nahi du: 1. **Startup teknologikoa (15 langile)** — datu-zientzialari bat, malgutasun handia behar, aurrekontu txikia. 2. **Banketxe handia (5.000 langile)** — sektore arautua, GDPR, auditoretza-beharra. 3. **Ikerketa-zentro biomedikoa** — biologoak (ez programatzaileak), datu-multzo konplexuak. **Eginkizuna:** Talde bakoitzak enpresa bat hautatu eta plataforma bat (edo konbinazioa) gomendatu, arrazoituta. Erabili 5.6 ataleko taula. Aurkeztu ikaskideei: zein plataforma, zergatik, eta zein abantaila/desabantaila. **Eztabaidatzeko:** Zenbat enpresak hautatu dute "Python soilik"? Zenbatek "biak"? Zergatik?

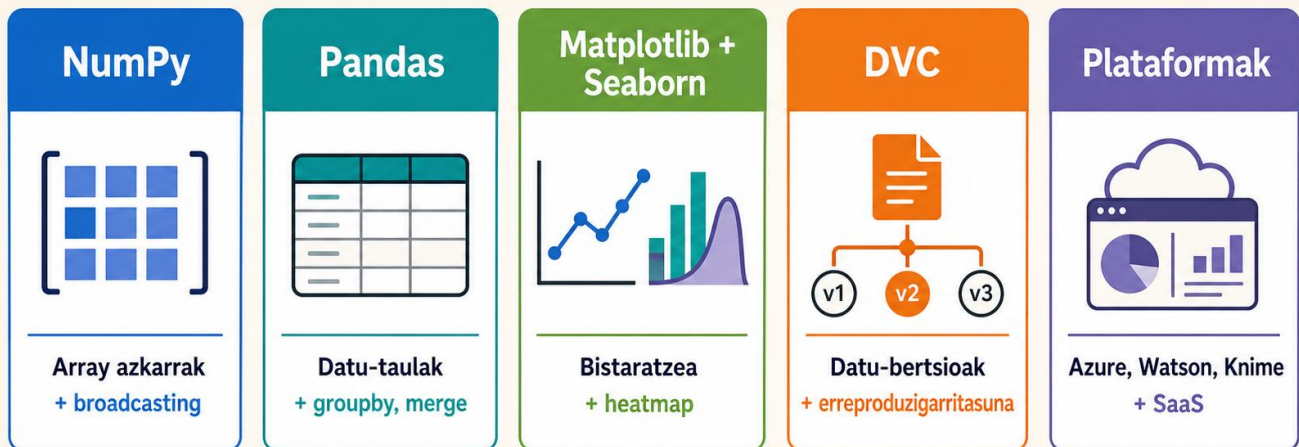
6. Laburpena eta Gakoak

Gai honetan datu-zientzialari batek egunero erabiltzen duen tresna-multzoa landu dugu, sarrerako salmenta-txostenaren eszenatik abiatuta. Hauek dira gaiaren bost zutabeak:

1. **NumPy** — zenbakizko kalkuluaren oinarria. Arrayak (mota homogeneoa, C-z idatzia), eragiketa bektorialak eta broadcasting (begiztarik gabe), indexazioa eta boolean maskak, eta estatistika/algebra linealaren oinarriak. Liburutegi guztien azpian dago.
2. **Pandas** — datu-taulen kudeaketa. Series eta DataFrame, datuak kargatu iturri anizkoitzetatik (CSV, Excel, JSON, SQL), datu-garriketa (proiektuaren %60-80!), agregazioa (`groupby`, `pivot_table`) eta taulen batzea (`merge`, `concat`).
3. **Matplotlib eta Seaborn** — bistaratzea. Figure/Axes egitura, grafiko-mota egokia datuari (lerroa, barrak, sakabanaketa, histograma), Seaborn-en `hue` eta korrelazio-heatmap-ak, eta esportazioa (PNG, PDF, SVG).
4. **DVC** — datuen bertsio-kontrola. "Datu-zientzialarientzako Git": datu handien puntero txikiak Giten, datuak kanpoko biltegian. Esperimentu-erreproduzigarritasuna.
5. **Plataforma komertzialak** — Azure ML, IBM Watson, Knime, SPSS. Open source eta SaaS osagarriak dira; bakoitzak bere testuingurua du. Bietan moldatzea balio handikoa da lan-merkatuan.

2. gaiaren bost zutabeak

Datu-zientzialariaren eguneroko tresnak



Hauek dira ML eta GenAI gaien oinarria

21. Irudia: 2. gaiaren bost zutabeak laburtuta — NumPy, Pandas, Matplotlib/Seaborn, DVC eta plataforma komertzialak, bakoitza bere ikono eta gako-kontzeptuekin.

🔑 **Gako-kontzeptuak** - **NumPy array** = mota homogeneousko bloke trinkoa, 10-100 aldiz azkarragoa zerrendak baino. - **Broadcasting** = forma desberdineko array-en arteko eragiketa begiztarik gabe. - **Boolean maska** = baldintza bat → True/False → datuak iragazi. - **DataFrame** = Pandas-en taula, zutabe bakoitza Series bat. - **Datu-garbiketa** = proiektuaren denboraren %60-80; "garbage in, garbage out". - **groupby** = split-apply-combine (SQL GROUP BY bezala). - **merge** = SQL JOIN; JOIN-mota egokia kritikoa da. - **Korrelazio-heatmap** = ML-ren aurretik ezaugarri garrantzitsuak ikusteko. - **DVC** = Git datuentzat; erreproduzigarritasuna. - **Open source vs SaaS** = osagarriak, ez aurkari.

Datu-zientziaren stack hau hurrengo gaietako oinarria da: ML ingeniartzan (3. gaia) datu garbi eta ondo prestatuak behar dira ereduak entrenatzeko, eta GenAI integrazioan (4. gaia) ere datu-prozesamendua funtsezkoa da. Hemen ikasitakoa egunero erabiliko duzu datu-zientzialari edo ML-ingeniari gisa.

7. Erreferentziak

Dokumentazio Ofizialak

- [NumPy dokumentazioa](https://numpy.org/doc/stable/)
- [Pandas dokumentazioa](https://pandas.pydata.org/docs/)
- [Matplotlib dokumentazioa](https://matplotlib.org/stable/contents.html)
- [Seaborn dokumentazioa](https://seaborn.pydata.org/)
- [DVC dokumentazioa](https://dvc.org/doc)
- [Azure Machine Learning dokumentazioa](https://learn.microsoft.com/azure/machine-learning/)
- [IBM Watson Studio dokumentazioa](https://www.ibm.com/products/watson-studio)
- [Knime Analytics Platform](https://www.knime.com/)

Tutorialak eta Baliabide Osagarriak

- [NumPy: 100 ariketa](https://github.com/rougier/numpy-100)
- [Pandas: 100 puzzle](https://github.com/ajcr/100-pandas-puzzles)
- [Python Data Science Handbook (doakoa)](https://jakevdp.github.io/PythonDataScienceHandbook/)
- [Kaggle: Pandas kurtsoa (doakoa)](https://www.kaggle.com/learn/pandas)
- [From Data to Viz — grafiko mota aukeratzeko gida](https://www.data-to-viz.com/)
- [Seaborn: adibide-galeria](https://seaborn.pydata.org/examples/index.html)

Datu-multzo Praktikoak

- [Kaggle datu-multzoak](https://www.kaggle.com/datasets)
- [UCI Machine Learning Repository](https://archive.ics.uci.edu/)
- [Seaborn barne-datu-multzoak](https://github.com/mwaskom/seaborn-data):
`sns.load_dataset("tips"), sns.load_dataset("iris"), sns.load_dataset("penguins")...`

Araudia

- Espainiako Errege Dekretua **279/2021**, apirilaren 20koa, Adimen Artifizial eta Big Data espezializazio-ikastaroa ezartzen duena (BOE). RA1 eta RA2 ikaskuntza-emaitezak.

5073 Modulua — Adimen Artifizialaren Programazioa | 2. Gaia: Datu Zientziaren Stack-a | Andoni Elola | LaneKi — FP Euskadi | 2025–2026